



Implementation-of-RISC-V 32-bit Single & Multi Cycle design

Verilog RTL Design — Full Technical Report

Module-by-Module Description

Internal Operation, and Datapath Explanation

Teamwork:

Mostafa Hosny

Hosam Fawzy

Doaa Ali

Under Supervision: Eng. Mohamed Salah

CONTENTS

1.	Introduction.....	3
2.	Single Cycle RISC_V 32-bit.....	5
2.1	Design Files	6
2.2	Implementation	7
2.3	Building Blocks	8
2.4	Verification.....	21
3.	Multi-Cycle RISC-V 32-bit	28
3.1	System Overview	28
3.2	Design Files -- Module Inventory	28
3.3	Implementation	29
3.4	Building Blocks	31
3.5	Verification.....	40
4.	Conclusion	50
5.	ABP bus and Address decoder	51
5.1	Ports Description.....	51
5.2	FSM State Machine.....	53
5.3	Data Flow & Logic	54
5.4	Test Cases Executed.....	54
6.	Address decoder.....	55
6.1	Ports Description.....	55
6.2	Internals & Flow	56
7.	I2C Master Peripheral	56
7.1	System Overview	56
7.2	Implementation	57
7.3	Building Blocks	59
7.4	Verification.....	68
8.	Future Work	70
9.	Conclusion	70

1. Introduction

RISC-V is an open standard instruction set architecture (ISA) based on reduced instruction set computing (RISC) principles. Unlike proprietary ISAs, RISC-V is freely available and can be used for various applications, from small embedded systems to powerful supercomputers. The RISC-V instruction set architecture (ISA) is designed with simplicity and efficiency in mind. This simplicity offers several advantages, particularly in terms of understanding, implementation, and optimization. RISC-V includes a minimal set of instructions necessary for efficient computation, avoiding the complexity of numerous specialized instructions found in other ISAs. The base integer instruction set (RV32I) consists of only about 40 instructions, covering essential operations like arithmetic, logic, control flow, and memory access. RISC-V instructions are uniformly 32 bits long, simplifying instruction decoding. There are no variable-length instructions, which reduces complexity. RISC-V also supports a compressed instruction set, known as the RISC-V Compressed (RVC) extension, which includes 16-bit instructions. Here is an overview of RISC-V and its key features:

Simplicity and Modularity: RISC-V has a simple and clean design with a small set of base instructions. This simplicity makes it easier to implement and understand. It is modular, meaning it includes a base set of instructions that can be extended with optional instruction sets for specific needs, such as floating-point operations or vector processing.

Open Standard: RISC-V is open and royalty-free, which means anyone can use, modify, and implement it without paying licensing fees. This encourages innovation and collaboration in the hardware community.

Scalability: RISC-V can be used in a wide range of devices, from small microcontrollers to large-scale processors in data centers. Its scalability makes it suitable for many applications.

Customizability: Designers can customize RISC-V processors by adding their instructions tailored to specific applications. This flexibility allows for optimized performance and efficiency.

Compatibility and Ecosystem: RISC-V maintains compatibility with existing software tools and operating systems, fostering a growing ecosystem of

development tools, libraries, and support resources. It supports a variety of operating systems, including Linux, making it versatile for different applications.

Energy Efficiency: The RISC-V architecture is designed to be energy efficient, making it ideal for battery-powered devices and other applications where power consumption is critical.

Security Features: RISC-V includes provisions for implementing security features such as secure boot and cryptographic extensions, enhancing the security of devices using this architecture.

Community and Support: The RISC-V Foundation, a nonprofit organization, oversees the development and promotion of the RISC-V ISA. The foundation includes members from academia, industry, and the open-source community, ensuring broad support and continuous improvement of the architecture.

Examples of RISC-V Use Cases :

Embedded Systems: RISC-V is popular in embedded systems due to its simplicity and efficiency.

Internet of Things (IoT): Its energy efficiency and scalability make RISC-V a good fit for IoT devices.

High-Performance Computing: With extensions and customizations, RISC-V can be used in high-performance computing environments.

Educational Tools: RISC-V's open nature makes it an excellent tool for teaching computer architecture and processor design.

RISC-V represents a significant shift in the world of processor design with its open, scalable, and customizable approach. Its simplicity, energy efficiency, and robust ecosystem make it a compelling choice for a wide range of applications, driving innovation in both academic and commercial sectors.

2. Single Cycle RISC_V 32-bit

A **Single-Cycle RISC-V Processor** executes every instruction in exactly **one clock cycle**. This means the total time required for an instruction to fetch from memory, decode registers, execute through the ALU, access data memory, and write back the result is constrained to a single rising edge of the system clock.

Advantages and disadvantages

Advantages

- Simple architecture
- Easy to understand
- Easy to implement in Verilog
- No pipeline hazards
- Good for learning CPU architecture
- Straightforward control logic

Disadvantages

- Poor maximum frequency
- Long critical path
- Every instruction uses the same clock period
- Hardware resources may be underutilized
- Not suitable for high-performance processors

This report documents a single-cycle RISC-V (RV32I subset) processor implemented in Verilog. In a single-cycle design, every instruction regardless of type completes fetch, decode, execute, memory access, and register write-back within one clock cycle. This makes the control logic simple (no pipeline hazards, no stalling), at the cost of a clock period long enough to accommodate the slowest instruction path (typically a load instruction, which passes through instruction memory, register file, ALU, and data memory in the same cycle).

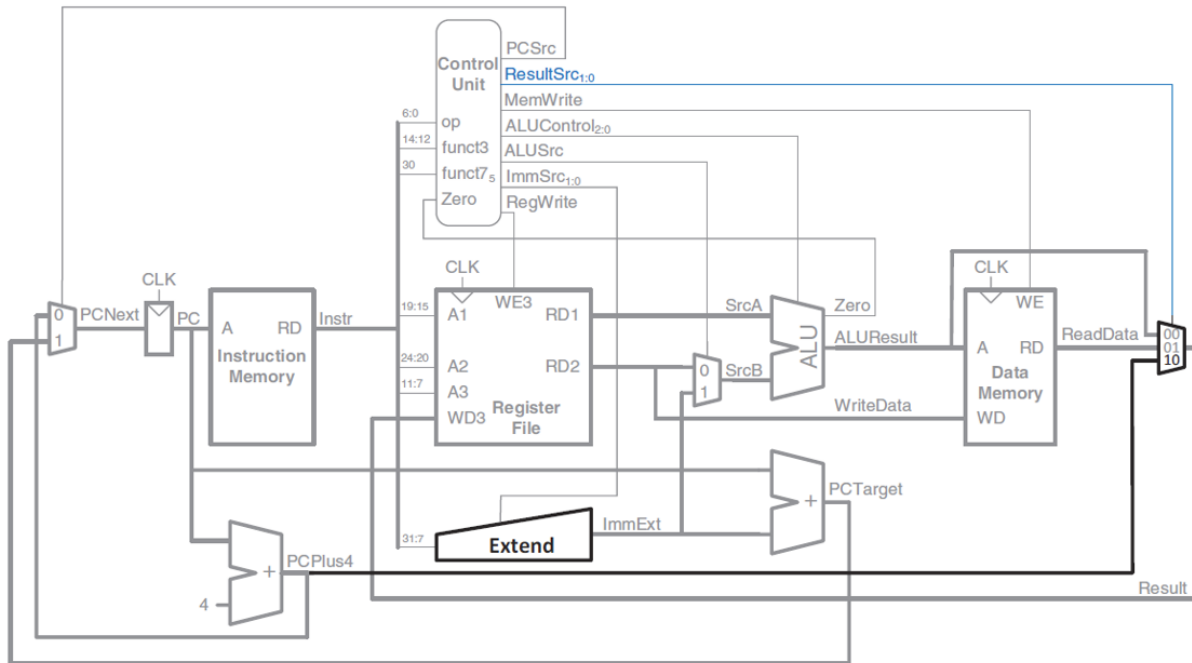
The design supports the following instructions: `addi`, `add`, `sub`, `and`, `or`, `slt`, `lw` (load word), `sw` (store word), `beq` (branch if equal), and `jal` (jump and link).

2.1 Design Files

- **riscv_single_cycle.v** : top-level module wiring the entire datapath and control path together
- **PC.v** : Program Counter register
- **PC_plus_4.v** : PC increment adder
- **PC_Target_Adder.v** : branch/jump target address adder
- **Inst_Mem.v** : Instruction Memory (ROM-like instruction storage)
- **Reg_Files.v** : Register Files (32 x 32-bit general-purpose register file)
- **Immediate_Generator.v** : sign-extended immediate generator
- **ALU.v** : Arithmetic Logic Unit
- **MUX_2x1.v** : parameterized 2-to-1 multiplexer
- **MUX_3x1.v** : parameterized 3-to-1 multiplexer
- **Data_Memory.v** : Data_Memory (RAM for load/store)
- **Control_Unit.v** : Main decoder + ALU decoder (instruction decode / control signal generation)
- **Riscv_Single_Cycle_TB.v** : self-checking testbench exercising all supported instructions

2.2 Implementation

Block Diagram for the Whole Design



High-Level Datapath

The processor follows the classic Harris & Harris single-cycle RISC-V datapath structure. On every rising clock edge the Program Counter updates to a new address. That address feeds the Instruction Memory, which returns the current instruction. The instruction's fields select register operands from the Register File and are also decoded by the Control Unit to produce every control signal in the datapath. The Immediate Generator extracts and sign-extends the instruction's constant field, the ALU computes the required arithmetic/logic/address result, the Data Memory handles loads and stores, and a result multiplexer chooses what is written back into the register file. In parallel, adders compute `PC+4` (sequential next instruction) and `PC+immediate` (branch/jump target), and a mux chooses between them based on whether a branch is taken.

2.3 Building Blocks

1) Program Counter (PC)

Holds the address of the instruction currently being executed. It is the only truly stateful element that drives instruction fetch, and it is updated once per clock cycle.

Port	Dir	Width	Description
clk	input	1	Clock — PC updates on the rising edge.
rst	input	1	Active-low asynchronous reset.
pc_in	input	32	Next PC value, supplied by the pc_mux (PCPlus4 or PCTarget).
pc_out	output	32	Current PC value, fed to instruction memory and the target adder.

2) PC_plus_4

Computes the address of the next sequential instruction. RISC-V instructions are 4 bytes (one word) long, so the default next address is simply the current PC plus 4.

Port	Dir	Width	Description
pc_plus_4_in	input	32	Current PC value.
pc_plus_4_out	output	32	PC + 4, used both as the default next-PC and as the return address for jal.

3) PCTarget_Adder Branch/Jump Target Adder

Computes the target address for branch and jump instructions using PC-relative addressing, as defined by the RISC-V ISA: $\text{target} = \text{PC} + \text{sign-extended immediate}$.

Port	Dir	Width	Description
pc_in	input	32	Current PC value.
imm_ext	input	32	Sign-extended immediate from Imme_Gen (B-type for beq, J-type for jal).
pc_target	output	32	$\text{PC} + \text{imm_ext}$ — candidate next PC used when a branch/jump is taken.

4) Instruction_Memory

Stores the program's machine code and supplies the instruction addressed by the current PC to the rest of the datapath.

Port	Dir	Width	Description
clk	input	1	Clock, used only for the synchronous reset-clear loop.
rst	input	1	Active-low asynchronous(-style) reset that clears memory contents.
inst_mem_read_addr	input	32	Address used to index the instruction array (connected directly to PC).
inst_mem_out	output	32	The fetched 32-bit instruction word.

Internal Storage

```
reg [31:0] inst_mem [0:63]; // 64 words of 32-bit
instruction memory
```

This declares a memory array of 64 entries, each 32 bits wide.

5) Register_Files

Implements the RISC-V general-purpose register file: 32 registers (x0–x31), each 32 bits wide, with two simultaneous read ports and one write port, matching what a single-cycle datapath needs (two source operands and one destination write, all in the same cycle).

Port	Dir	Width	Description
clk	input	1	Clock — writes occur on the rising edge.
rst	input	1	Active-low asynchronous reset — clears the register array.
reg_write_addr	input	5	Destination register number, from instruction bits [11:7] (rd).
reg_read_addr1	input	5	First source register number, from instruction bits [19:15] (rs1).
reg_read_addr2	input	5	Second source register number, from instruction bits [24:20] (rs2).
reg_write_data	input	32	Data to write into reg_write_addr, driven by the write-back Result.
reg_write_en	input	1	RegWrite — when 1, the write actually occurs on the clock edge.
reg_read_data1	output	32	Combinational read of register reg_read_addr1.
reg_read_data2	output	32	Combinational read of register reg_read_addr2.

6) Imme_Gen --- Immediate Generator

RISC-V encodes immediate (constant) values differently depending on the instruction format, scattering the bits across the instruction word to keep register fields in fixed positions. This module extracts the correct bits for the current instruction type and sign-extends them to a full 32-bit value

Port	Dir	Width	Description
instr	input	32	The full fetched instruction word.
ImmSrc	input	2	Format selector from the Control_Unit (00 = I, 01 = S, 10 = B, 11 = J).
ImmExt	output	32	The sign-extended immediate value.

ImmSrc	ImmExt	Type	Description
00	{{20{Instr[31]}}, Instr[31:20]}	I	12-bit signed immediate
01	{{20{Instr[31]}}, Instr[31:25], Instr[11:7]}	S	12-bit signed immediate
10	{{20{Instr[31]}}, Instr[7], Instr[30:25], Instr[11:8], 1'b0}	B	13-bit signed immediate

```

always @(*) begin
    case (ImmSrc)
        // 2'b00: I-Type (e.g., lw, addi, jalr)
        2'b00: ImmExt = {{20{instr[31]}}, instr[31:20]};

        // 2'b01: S-Type (e.g., sw)
        2'b01: ImmExt = {{20{instr[31]}}, instr[31:25], instr[11:7]};

        // 2'b10: B-Type (e.g., beq, bne)
        2'b10: ImmExt = {{20{instr[31]}}, instr[7], instr[30:25], instr[11:8], 1'b0};

        // 2'b11: J-Type (e.g., jal)
        2'b11: ImmExt = {{12{instr[31]}}, instr[19:12], instr[20], instr[30:21], 1'b0};

        default: ImmExt = 32'bx;
    endcase
end

```

I-type (addi, lw, jalr)

Bits [31:20] of the instruction form a straightforward 12-bit signed immediate, sign-extended by replicating bit 31 (the instruction's sign bit) 20 times.

S-type (sw)

Because rd's field position must instead carry part of the store address offset, the 12-bit immediate is split into instr[31:25] (upper 7 bits) and instr[11:7] (lower 5 bits), reassembled and sign-extended the same way.

B-type (beq)

Branch offsets are encoded to represent even byte offsets (bit 0 is implicitly 0, since instructions are word/half-word aligned), so the hardware reconstructs a 13-bit signed offset from instr[7], instr[30:25], instr[11:8], appends a literal 0 bit, and sign-extends.

J-type (jal)

Similarly, jal's 21-bit signed offset is reconstructed from instr[19:12], instr[20], instr[30:21], with an implicit trailing 0 bit, then sign-extended from bit 31 across the top 12 bits.

If ImmSrc has any other value, ImmExt is driven to all-X (unknown) as a safeguard/placeholder for unused encodings — this never occurs in normal operation since the Control_Unit only issues the four defined codes.

7) ALU — Arithmetic Logic Unit

Performs the arithmetic and logic operation required by the current instruction on its two operands, and reports whether the result is zero (used for branch comparisons).

Port	Dir	Width	Description
SrcA	input	32	First operand — always the rs1 register value (RD1).

Port	Dir	Width	Description
SrcB	input	32	Second operand — either rs2 (RD2) or the sign-extended immediate, chosen upstream by ALUSrc.
ALUControl	input	3	Operation select code from the Control_Unit.
ALUResult	output	32	Result of the selected operation.
Zero	output	1	Asserted (1) when ALUResult equals zero.

Operation Table

ALUControl	Mnemonic	Operation
000	ADD	$\text{ALUResult} = \text{SrcA} + \text{SrcB}$ (used by add, addi, lw, sw address calc)
001	SUB	$\text{ALUResult} = \text{SrcA} - \text{SrcB}$ (used by sub, and by beq for equality test)
010	AND	$\text{ALUResult} = \text{SrcA} \& \text{SrcB}$ (used by and, andi)
011	OR	$\text{ALUResult} = \text{SrcA} \text{SrcB}$ (used by or, ori)
101	SLT	$\text{ALUResult} = 1$ if $\text{SrcA} < \text{SrcB}$ (signed), else 0 (used by slt, slti)

(Codes 100, 110, 111 are unused by this instruction subset and fall into the default case, producing an unknown/X result.)

8) MUX_2x1

A generic, width-parameterized 2-input multiplexer, reused twice in the top module: once to choose the next PC value, and once to choose the ALU's second operand.

Port	Dir	Width	Description
d0	input	WIDTH	Selected when s = 0.
d1	input	WIDTH	Selected when s = 1.
s	input	1	Select line.
y	output	WIDTH	$y = s ? d1 : d0$.

9) MUX_3x1

Selects the value written back to the register file at the end of instruction execution, among three possible sources.

Port	Dir	Width	Description
d0	input	WIDTH	Selected when s = 2'b00 — the ALU result.
d1	input	WIDTH	Selected when s = 2'b01 — data read from memory.
d2	input	WIDTH	Selected when s = 2'b10 — PC + 4 (return address).
s	input	2	ResultSrc select line.
y	output	WIDTH	Selected output, written back to the register file.

10) **Data Memory — Data_Memory**

Implements the data RAM used by load (lw) and store (sw) instructions.

Port	Dir	Width	Description
clk	input	1	Clock — writes occur on the rising edge.
rst	input	1	Active-low asynchronous reset — clears all memory contents.
WE	input	1	Write Enable (driven by MemWrite) — 1 during a store.
A	input	32	Address, driven by ALUResult (the computed effective address).
WD	input	32	Write Data — the value to store (WriteData / RD2).
RD	output	32	Read Data — combinational read of memory[A].

11) **Control_Unit**

The Control_Unit is the decode-stage brain of the processor. It looks at the instruction's opcode and function bits and produces every control signal that steers the datapath: which immediate format to use, which ALU operand source and operation to use, whether to write memory or the register file, what to write back, and whether to redirect the PC for a branch or jump. It is split into two cooperating pieces: a Main Decoder (instruction-type based) and an ALU Decoder (fine-grained operation based).

Port	Dir	Width	Description
op	input	7	Instruction opcode, Instr[6:0].
funct3	input	3	Instr[14:12] — distinguishes operations within the same opcode.
funct7_5	input	1	Instr[30] — the single funct7 bit that distinguishes add/sub and srl/sra.
Zero	input	1	Zero flag from the ALU, used for conditional branch resolution.
PCSrc	output	1	Selects PCTarget instead of PCPlus4 when asserted (Branch & Zero).
ResultSrc	output	2	Selects the write-back source for the result_mux.
MemWrite	output	1	Enables a data memory store.
ALUSrc	output	1	Selects register vs. immediate as the ALU's second operand.
ImmSrc	output	2	Selects the immediate format for Imme_Gen.
RegWrite	output	1	Enables a register file write.
ALUControl	output	3	Selects the ALU's operation.

Main Decoder — Instruction Recognition

Based purely on the 7-bit opcode, the main decoder sets every control signal except the exact ALU operation (which it defers to a 2-bit intermediate code, ALUOp, resolved later by the ALU decoder). The recognized opcodes are:

Opcode	Mnemonic	Format	Control Signals Set
0000011	lw	I	RegWrite=1, ALUSrc=1, MemWrite=0, ResultSrc=01 (mem), ALUOp=00 (add)
0100011	sw	S	RegWrite=0, ALUSrc=1, MemWrite=1, ALUOp=00 (add)
0110011	R-type	R	RegWrite=1, ALUSrc=0, ResultSrc=00 (ALU), ALUOp=10 (decode via funct3/funct7)
1100011	beq	B	RegWrite=0, ALUSrc=0, Branch=1, ALUOp=01 (subtract, for equality test)
0010011	addi	I	RegWrite=1, ALUSrc=1, ResultSrc=00 (ALU), ALUOp=10
1101111	jal	J	RegWrite=1, ResultSrc=10 (PC+4), Branch=1 (forces PCSrc via Branch & Zero)

Any opcode not in this list falls into the default branch, which safely de-asserts RegWrite and MemWrite (a no-operation state).

PCSrc generation

```
assign PCSrc = Branch & Zero;
```

PCSrc is a simple AND of the internal Branch flag and the ALU's Zero output. For beq, Branch = 1 is set only when the opcode is the branch opcode, and the ALU is configured (via ALUOp = 01 -> ALUControl = SUB) to compute $rs1 - rs2$, so Zero = 1 exactly when $rs1$ equals $rs2$ — this is how a conditional branch's condition is evaluated using the same ALU that performs arithmetic. For jal, Branch is also forced to 1 unconditionally (since jal is an unconditional jump and must always

redirect the PC), relying on Zero also evaluating to 1 for that case (see the design note in Section 13.5 below regarding this reuse).

ALU Decoder — Resolving the Exact Operation

R-type and I-type ALU instructions share opcodes across several distinct operations (add/sub, and, or, slt), so the main decoder only narrows things down to $ALUOp = 2'b10$ ("look at funct3/funct7 to decide"), leaving the final decode to a second case statement:

```
case (ALUOp)
  2'b00: ALUControl = 3'b000; // Addition (lw, sw)
  2'b01: ALUControl = 3'b001; // Subtraction (beq)
  2'b10: begin
    case (funct3)
      3'b000: begin
        // Check sub vs add (R-type sub uses funct7[5] == 1)
        if ({op[5], funct7_5} == 2'b11)
          /* op[5] distinguishes R-type instructions (op = 7'b0110011, so op[5] = 1)
             from I-type instructions like addi (op = 7'b0010011, so op[5] = 0).
             funct7_5 (bit 30 of the instruction) distinguishes sub (funct7[5] = 1)
             from add (funct7[5] = 0). */
          ALUControl = 3'b001; // sub
        else
          ALUControl = 3'b000; // add / addi
        end
      3'b010: ALUControl = 3'b101; // slt / slti
      3'b110: ALUControl = 3'b011; // or / ori
      3'b111: ALUControl = 3'b010; // and / andi
      default: ALUControl = 3'bxxx;
    endcase
  end
default: ALUControl = 3'bxxx;
endcase
```

The add/sub distinction is the trickiest case: both add and addi use $funct3 = 000$, but only the R-type sub instruction (opcode 0110011, $funct7$ bit 30 = 1) should subtract — addi (opcode 0010011) always uses $funct3 = 000$ too, but must always add, never subtract, regardless of instruction bit 30 (which has a different meaning in I-type encoding). The design resolves this by checking $\{op[5], funct7_5\}$: $op[5]$ is 1 only for the R-type opcode (0110011) and 0 for the I-type opcode (0010011), so the concatenation only equals $2'b11$ when the instruction is truly R-type sub. This correctly prevents addi from ever being misinterpreted as a subtraction.

12) riscv_single_cycle — Top-Level Module

This is the top-level structural module. It contains no behavioral logic of its own instead it instantiates every functional block (PC, adders, memories, register file, immediate generator, ALU, muxes, and control unit) and connects them together with wires that exactly mirror the standard single-cycle RISC-V datapath diagram.

Port	Dir	Width	Description
clk	input	1	System clock. All sequential elements (PC, register file, instruction memory, data memory) update on its rising edge.
rst	input	1	Active-low asynchronous reset. When driven low, PC, register file, instruction memory, and data memory are cleared to zero.

Structural Wiring — How the Instruction Flows

Step 1 — Fetch

A 2-to-1 mux (pc_mux) chooses PCNext: either PCPlus4 (sequential execution) or PCTarget (branch/jump), selected by PCSrc.

The Program_Counter register (pc_inst) latches PCNext into PC on the clock edge. PC is fed to Instruction_Memory (imem), which outputs the current instruction Instr. In parallel, PC_plus_4 (pc_add4) computes PC+4, and PCTarget_Adder (pc_target_add) computes PC + ImmExt (the branch/jump target).

Step 2 — Decode / Register Read

Register_Files (rf) is read using instruction fields Instr[19:15] and Instr[24:20] as the two source register addresses, and Instr[11:7] as the destination register address for write-back.

The two read values become RD1 (assigned to SrcA) and RD2 (assigned to WriteData). Simultaneously, Imme_Gen decodes Instr according to ImmSrc

(supplied by the Control_Unit) to produce the sign-extended immediate ImmExt, and Control_Unit decodes Instr[6:0] (opcode), Instr[14:12] (funct3), and Instr[30] (funct7 bit 5) into every control signal.

Step 3 — Execute

A second 2-to-1 mux (srcb_mux) selects the ALU's second operand SrcB: either WriteData (register-register operations, branches) or ImmExt (immediate operations, loads, stores), controlled by ALUSrc. The ALU computes ALUResult = f(SrcA, SrcB, ALUControl) and produces the Zero flag (asserted when ALUResult == 0), which the Control_Unit uses to decide whether a conditional branch is taken.

Step 4 — Memory

Data_Memory (dmem) is addressed by ALUResult. On a store (MemWrite = 1) it writes WriteData into that address on the clock edge; it always combinationally outputs the current contents of that address as ReadData (used for loads).

Step 5 — Write-back

A 3-to-1 mux (result_mux) selects what gets written back into the register file as Result: ALUResult (arithmetic instructions), ReadData (loads), or PCPlus4 (jal, which stores the return address). ResultSrc, generated by the Control_Unit, drives the selection. Result is fed back into Register_Files as reg_write_data, gated by RegWrite.

2.4 Verification

13) Riscv_Single_Cycle_TB

A self-checking testbench that instantiates the full processor, preloads a small hand-written program directly into instruction memory, runs it for a fixed number of cycles, and automatically compares the resulting register and memory contents against expected values.

Structure

Generates a free-running clock with a 10 ns period (5 ns high, 5 ns low).

Defines three reusable checker tasks

- check_reg,
- check_mem,
- check_val

each of which increments a global check counter, compares an actual value to an expected one using Verilog's case-equality operator (!==, which also correctly distinguishes X/Z), and logs a PASS or FAIL line with the mismatch details.

A load_program task force-writes 16 machine-code words directly into dut.imem.inst_mem[...] (bypassing any real instruction loader), covering every supported instruction: addi, add, sub, and, or, slt, sw, lw, a taken beq, a not-taken beq, and jal.

The main initial block first exercises the asynchronous reset, verifying PC, a sample register, and a sample memory word are all zero immediately after reset.

It then releases reset, loads the program, and lets the design run for 16 clock edges one instruction retiring per cycle, consistent with the single-cycle architecture before checking every register and memory location the program was expected to modify.

A second reset is applied mid-simulation to confirm the design correctly returns to the all-zero state a second time (guarding against reset logic that only works once).

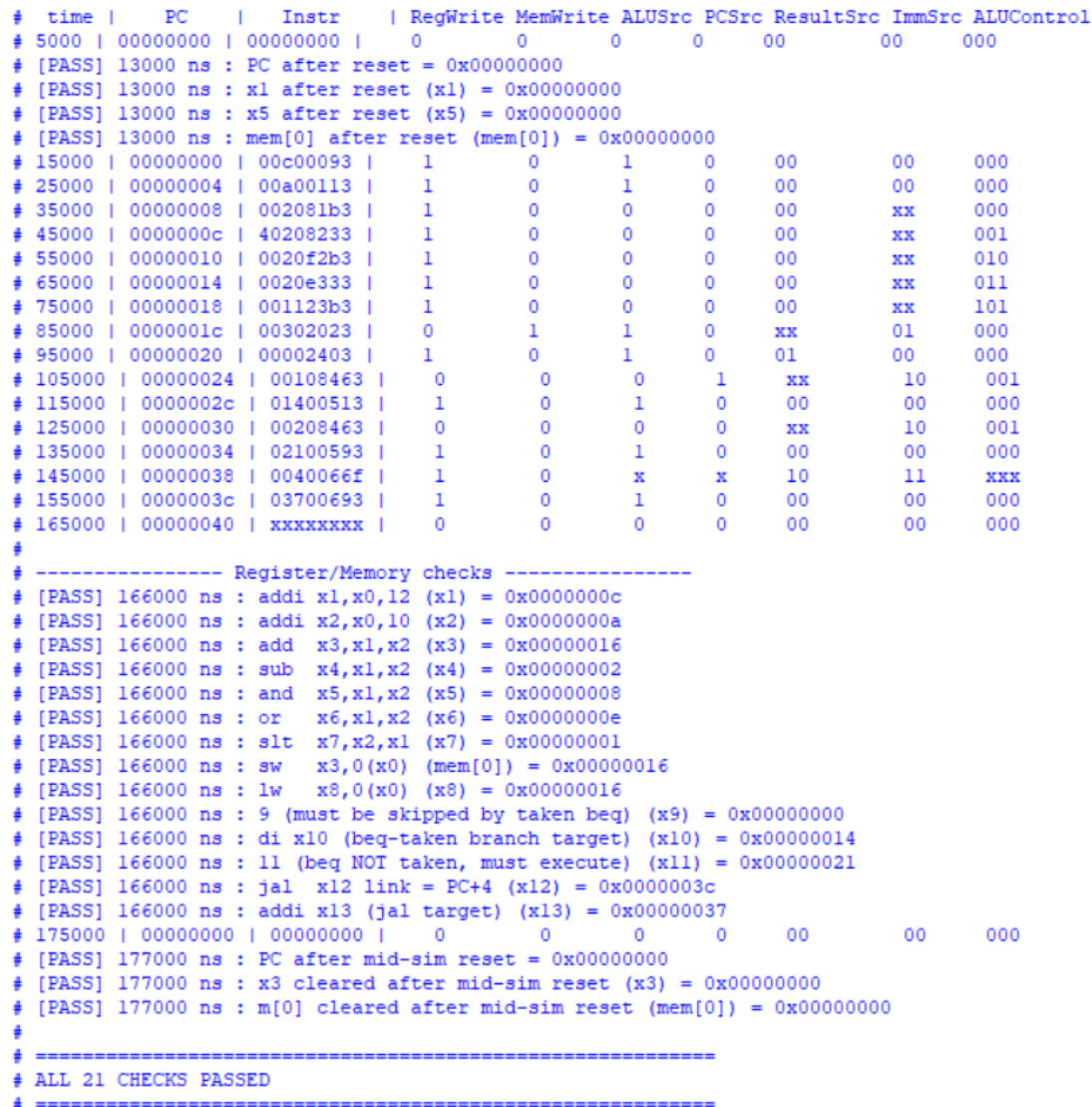
A background always block prints a per-cycle trace of PC, Instr, and every control signal, useful for waveform-free debugging.

Test Program

The **16-instruction** program packed into addresses 0–60 is deliberately built to exercise every opcode while staying inside the instruction memory's addressable window

Addr	Instruction	Hex Encoding	Effect
0	addi x1,x0,12	0x00c00093	x1 = 12
4	addi x2,x0,10	0x00a00113	x2 = 10
8	add x3,x1,x2	0x002081b3	x3 = 22
12	sub x4,x1,x2	0x40208233	x4 = 2
16	and x5,x1,x2	0x0020f2b3	x5 = 8
20	or x6,x1,x2	0x0020e333	x6 = 14
24	slt x7,x2,x1	0x001123b3	x7 = 1 (10<12)
28	sw x3,0(x0)	0x00302023	mem[0] = 22
32	lw x8,0(x0)	0x00002403	x8 = 22
36	beq x1,x1,8	0x00108463	taken, skip addr 40
40	addi x9,x0,99	0x06300493	skipped, x9 stays 0
44	addi x10,x0,20	0x01400513	branch target, x10 = 20
48	beq x1,x2,8	0x00208463	not taken (12 != 10)
52	addi x11,x0,33	0x02100593	executes, x11 = 33
56	jal x12,4	0x0040066f	x12 = 60 (link = PC+4)
60	addi x13,x0,55	0x03700693	jal target, x13 = 55

We Simulate the design with modelsim and check the flow with the transcript and the wave forms



Let's discuss how the core perform this 3 instructions in 3 clock cycle and show the result on the wave form

addr	instr	meaning
0	00c00093	addi x1, x0, 12
4	00a00113	addi x2, x0, 10
8	002081b3	add x3, x1, x2 -> 22

Instruction 1: addi x1, x0, 12 (0x00C00093)

1. Fetch Stage

- **Program Counter:** Outputs PC = 0.
- **Instruction Memory:** Reads address 0 and fetches Instr = 32'h00C00093.
- **Adder (PC_plus_4):** Computes PCPlus4 = 0 + 4 = 4.

2. Decode & Register Read Stage

- **Instruction Field Breakdown:**
 - op = Instr[6:0] = 7'b0010011 (I-Type ALU instruction).
 - funct3 = Instr[14:12] = 3'b000 (ADD operation).
 - rs1 = Instr[19:15] = 5'b00000 (x0).
 - rd = Instr[11:7] = 5'b00001 (x1).
- **Register File (Register_Files):** Reads register x0 on port RD1 (SrcA = 0).
- **Immediate Generator (Imme_Gen):** Decodes the I-type immediate instr[31:20], outputting ImmExt = 32'd12.
- **Control Unit (Control_Unit):** Sets control flags for an I-type addi instruction:
 - RegWrite = 1 (enable register write)
 - ALUSrc = 1 (select immediate ImmExt for ALU operand B)
 - ALUControl = 3'b000 (ADD)
 - MemWrite = 0 (disable data memory write)

- ResultSrc = 2'b00 (select ALU output for writeback)
- PCSrc = 0 (select PCPlus4 as next PC)

3. Execute Stage

- **ALU MUX (srcb_mux):** Selects ImmExt because ALUSrc = 1, driving SrcB = 12.
- **ALU (ALU):** Computes ALUResult = SrcA + SrcB = 0 + 12 = 12. Zero = 0.

4. Memory & Writeback Stage

- **Data Memory (Data_Memory):** Inactive (MemWrite = 0).
- **Result MUX (result_mux):** Selects ALUResult because ResultSrc = 2'b00, outputting Result = 12.
- **Clock Edge:** Register file writes Result = 12 into destination x1 (rd = 1). Program counter updates to PCNext = 4.

Instruction 2: addi x2, x0, 10 (0x00A00113)

1. Fetch Stage

- **Program Counter (Program_Counter):** Outputs PC = 4.
- **Instruction Memory (Instruction_Memory):** Reads address 4 and fetches Instr = 32'h00A00113.
- **Adder (PC_plus_4):** Computes PCPlus4 = 4 + 4 = 8.

2. Decode & Register Read Stage

- **Instruction Field Breakdown:**
 - op = Instr[6:0] = 7'b0010011 (I-Type ALU instruction).
 - funct3 = Instr[14:12] = 3'b000 (ADD operation).
 - rs1 = Instr[19:15] = 5'b00000 (x0).
 - rd = Instr[11:7] = 5'b00010 (x2).
- **Register File (Register_Files):** Reads register x0 on port RD1 (SrcA = 0).

- **Immediate Generator (Imme_Gen):** Decodes the I-type immediate $\text{instr}[31:20]$, outputting $\text{ImmExt} = 32'd10$.
- **Control Unit (Control_Unit):**
 - $\text{RegWrite} = 1$, $\text{ALUSrc} = 1$, $\text{ALUControl} = 3'b000$ (ADD), $\text{MemWrite} = 0$, $\text{ResultSrc} = 2'b00$, $\text{PCSrc} = 0$.

3. Execute Stage

- **ALU MUX (srcb_mux):** Selects ImmExt , driving $\text{SrcB} = 10$.
- **ALU (ALU):** Computes $\text{ALUResult} = \text{SrcA} + \text{SrcB} = 0 + 10 = 10$.

4. Memory & Writeback Stage

- **Data Memory (Data_Memory):** Inactive ($\text{MemWrite} = 0$).
- **Result MUX (result_mux):** Drives $\text{Result} = 10$.
- **Clock Edge:** Register file writes 10 into x2. Program counter updates to $\text{PCNext} = 8$.

Instruction 3: add x3, x1, x2 (0x002081B3)

1. Fetch Stage

- **Program Counter (Program_Counter):** Outputs $\text{PC} = 8$.
- **Instruction Memory (Instruction_Memory):** Reads address 8 and fetches $\text{Instr} = 32'h002081B3$.
- **Adder (PC_plus_4):** Computes $\text{PCPlus4} = 8 + 4 = 12$.

2. Decode & Register Read Stage

- **Instruction Field Breakdown:**
 - $\text{op} = \text{Instr}[6:0] = 7'b0110011$ (R-Type instruction).
 - $\text{funct3} = \text{Instr}[14:12] = 3'b000$.
 - $\text{funct7_5} = \text{Instr}[30] = 1'b0$ (ADD, not SUB).
 - $\text{rs1} = \text{Instr}[19:15] = 5'b00001$ (x1).
 - $\text{rs2} = \text{Instr}[24:20] = 5'b00010$ (x2).
 - $\text{rd} = \text{Instr}[11:7] = 5'b00011$ (x3).

- **Register File (Register_Files):**
 - Reads x1 on RD1 -> SrcA = 12 (written during cycle 1).
 - Reads x2 on RD2 -> WriteData = 10 (written during cycle 2).
- **Control Unit (Control_Unit):** Sets control flags for an R-type add instruction:
 - RegWrite = 1
 - ALUSrc = 0 (select register RD2 / WriteData for ALU operand B)
 - ALUControl = 3'b000 (ADD)
 - MemWrite = 0
 - ResultSrc = 2'b00
 - PCSrc = 0

3. Execute Stage

- **ALU MUX (srcb_mux):** Selects WriteData because ALUSrc = 0, driving SrcB = 10.
- **ALU (ALU):** Computes ALUResult = SrcA + SrcB = 12 + 10 = 22.

4. Memory & Writeback Stage

- **Data Memory (Data_Memory):** Inactive (MemWrite = 0).
- **Result MUX (result_mux):** Selects ALUResult, driving Result = 22.
- **Clock Edge:** Register file writes Result = 22 into destination register x3 (rd = 3). Program counter updates to PCNext = 12.
- **Final Register File State**
 - x1 = 12 in hexa = C
 - x2 = 10 in hexa = A
 - x3 = 22 in hexa = 16

3. Multi-Cycle RISC-V 32-bit

3.1 System Overview

This section documents a complete 32-bit RISC-V multi-cycle processor implemented in Verilog.

Unlike a single-cycle design, which finishes every instruction in one long clock period, the multi-cycle architecture breaks instruction execution into several shorter clock cycles (Fetch, Decode, Execute, Memory, Writeback).

Each cycle uses only the datapath hardware it actually needs, so the same ALU, the same memory port, and the same set of buses are reused across cycles instead of being duplicated.

This produces a smaller, simpler datapath at the cost of taking a variable number of cycles (3 to 5) to complete each instruction.

The design supports the following RISC-V instructions:

- R-type arithmetic/logic: add, sub, and, or, slt
- I-type arithmetic: addi
- Load/Store: lw (load word), sw (store word)
- Control flow: beq (branch if equal), jal (jump and link)

The processor is built from nine source files, each implementing one reusable hardware block. The top-level module, `riscv_multi_cycle`, instantiates and wires all of these blocks together; a separate testbench, `riscv_multi_cycle_tb`, drives the design with a small hand-assembled program and self-checks the register file and memory contents against expected results.

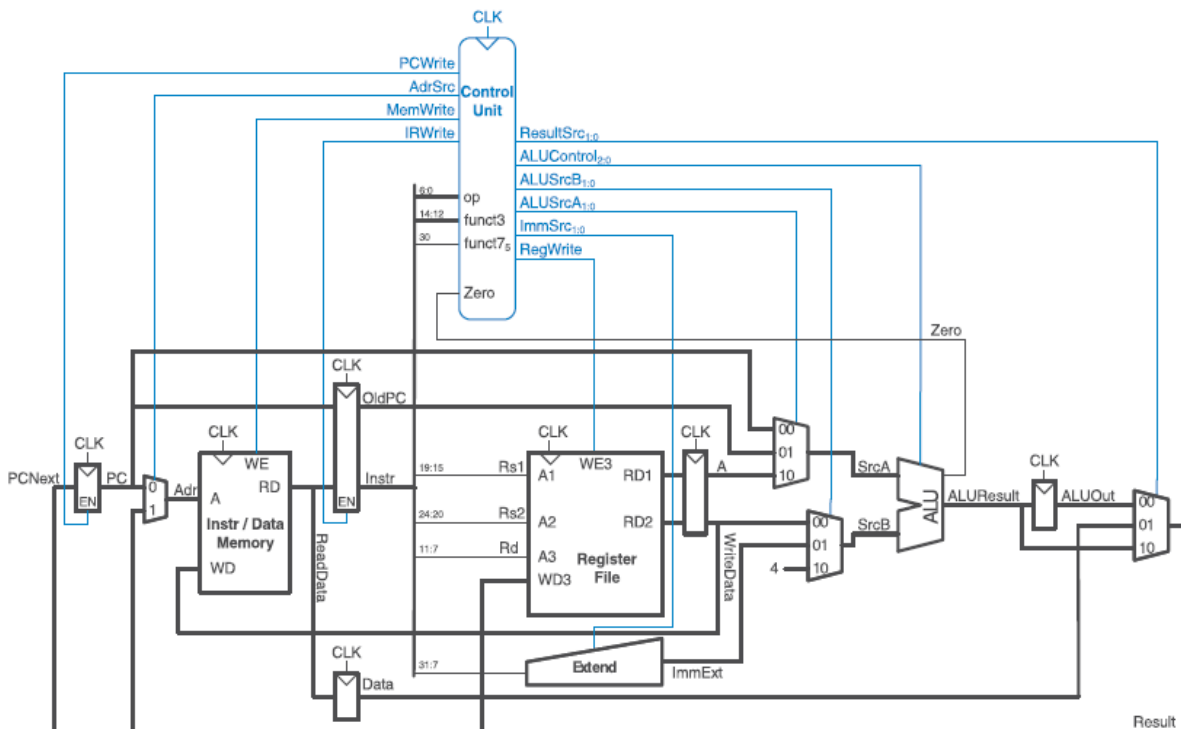
3.2 Design Files -- Module Inventory

- `ALU.v` Performs arithmetic/logic operations and produces the Zero flag used for branches.
- `Control_Unit.v` Finite State Machine that sequences each instruction through multiple cycles and generates every control signal in the datapath.
- `Immediate_Generaor.v` Sign-extends and reassembles the immediate field out of the raw instruction bits, according to instruction format.

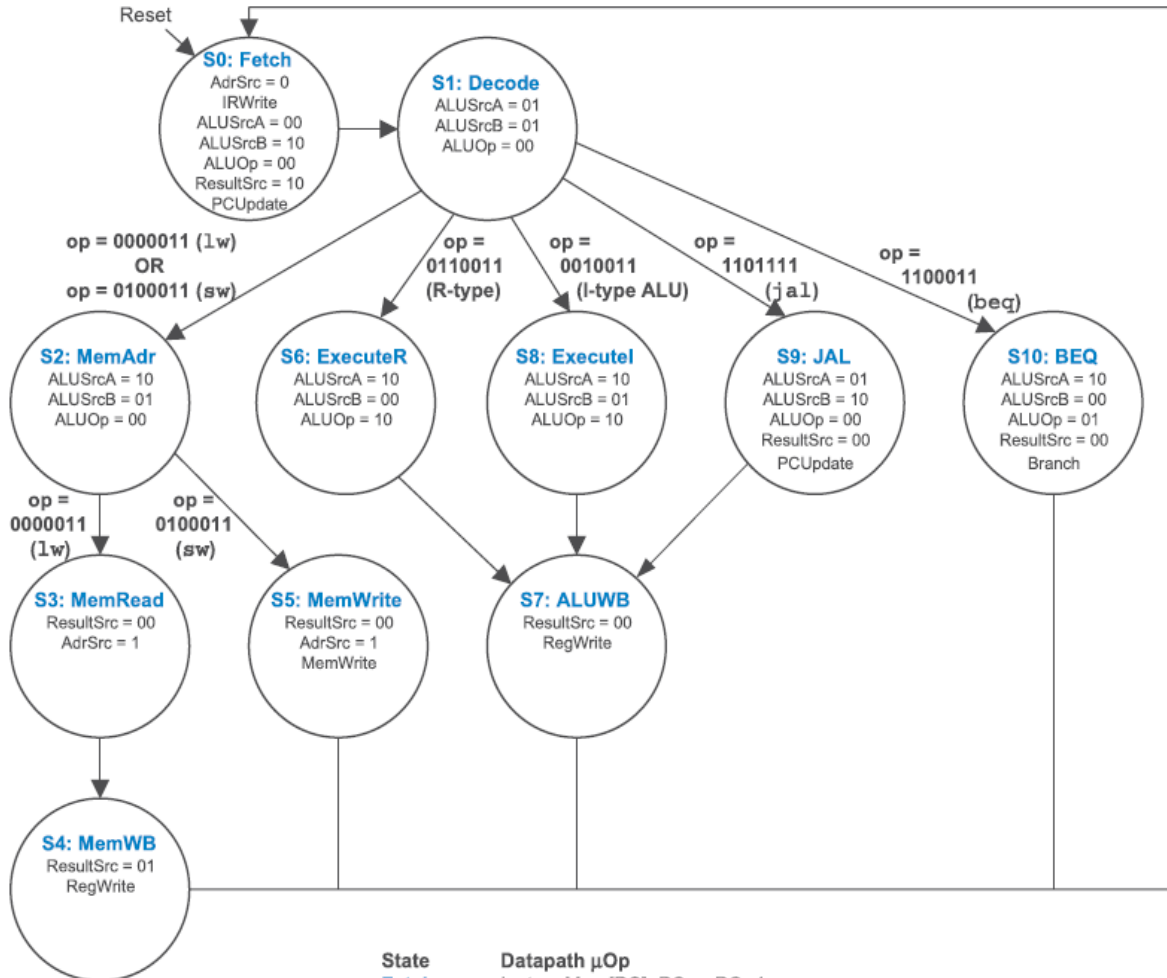
- `memory.v` Single unified 64-word memory used for both instructions and data, accessed one word at a time.
- `MUX_2x1.v` Generic parameterized 2-to-1 multiplexer used for simple binary signal selection
- `MUX_3x1.v` Generic parameterized 3-to-1 multiplexer used to select among three data sources (ALU operands, final result).
- `Reg_Files.v` 32 x 32-bit general-purpose register file with two read ports and one write port.
- `riscv_multi_cycle.v` Top-level module: instantiates all submodules and wires the complete multi-cycle datapath and control path together.
- `riscv_multi_cycle_tb.v` Testbench that generates the clock/reset, preloads a test program into memory, runs the simulation, and verifies results.

3.3 Implementation

Block Diagram for the Whole Design



Complete Multi Cycle Control FSM



3.4 Building Blocks

[1] ALU Module

The ALU (Arithmetic Logic Unit) is the computational heart of the datapath. It takes two 32-bit operands and, depending on a 3-bit control code, produces a 32-bit result plus a single-bit Zero flag. It is a purely combinational block it has no clock and no internal memory, so its output reacts immediately to any change on its inputs.

Port	Direction	Width	Description
SrcA	input	32	First ALU operand.
SrcB	input	32	Second ALU operand.
ALUControl	input	3	Operation select code driven by the Control Unit.
ALUResult	output (reg)	32	Result of the selected operation.
Zero	output (wire)	1	Asserted (1) when ALUResult equals zero; used for branch comparisons.

Operation Table

ALUControl	Operation	Behavior
3'b000	ADD	$ALUResult = SrcA + SrcB$
3'b001	SUB	$ALUResult = SrcA - SrcB$
3'b010	AND	$ALUResult = SrcA \& SrcB$ (bitwise)
3'b011	OR	$ALUResult = SrcA SrcB$ (bitwise)

ALUControl	Operation	Behavior
3'b101	SLT	ALUResult = 1 if SrcA < SrcB (signed comparison), else 0
other	undefined	ALUResult = 32'bx (don't-care / not used by this ISA subset)

[2] Control Unit Module

The Control Unit is the brain of the processor. It is a Moore-type Finite State Machine (FSM) that reads the opcode/funct fields of the currently latched instruction and, cycle by cycle, produces every control signal needed to steer the datapath: which multiplexer inputs to select, when to write the PC, register file, or memory, and what operation the ALU should perform. Because instructions in this design take a variable number of cycles, the Control Unit is what allows the same physical hardware (one ALU, one memory, one register file) to be reused differently on each cycle.

Port	Direction	Width	Description
clk	input	1	System clock.
rst	input	1	Active-low asynchronous reset.
op	input	7	Opcode field, instruction[6:0].
funct3	input	3	Function field, instruction[14:12].
funct7_5	input	1	Bit 30 of the instruction; distinguishes ADD from SUB, SRL from SRA, etc.

Port	Direction	Width	Description
Zero	input	1	Zero flag from the ALU; used to resolve branches.
PCWrite	output	1	Enables the Program Counter register to update.
AdrSrc	output	1	Selects memory address source: 0 = PC, 1 = ALU/Result register.
MemWrite	output	1	Enables a write into the unified memory.
IRWrite	output	1	Enables latching of the fetched word into the Instruction Register.
ResultSrc	output	2	Selects what feeds the register file / PC: ALUOut, memory Data, or the raw ALUResult.
ALUControl	output	3	Operation code forwarded to the ALU.
ALUSrcA	output	2	Selects ALU operand A: PC, OldPC, or register A.
ALUSrcB	output	2	Selects ALU operand B: register B, sign-extended immediate, or constant 4.
ImmSrc	output	2	Selects the immediate-decoding format (I / S / B / J).
RegWrite	output	1	Enables a write into the register file.

FSM State Encoding

State	Code	Meaning
FETCH	4'b0000	Fetch the instruction word from memory and increment PC by 4.
DECODE	4'b0001	Decode the instruction; pre-compute a possible branch/jump target (OldPC + Imm).
MEMADR	4'b0010	Compute the effective memory address (RegisterA + Imm) for lw/sw.
MEMREAD	4'b0011	Read the data word from memory (lw).
MEMWB	4'b0100	Write the loaded data back into the destination register (lw writeback).
MEMWRITE	4'b0101	Write RegisterB's value into memory (sw).
EXECUTER	4'b0110	Execute an R-type ALU operation (RegisterA op RegisterB).
ALUWB	4'b0111	Write the ALU result (held in ALUOut) back into the destination register.
EXECUTEI	4'b1000	Execute an I-type ALU operation (RegisterA op Immediate).
JAL	4'b1001	Compute the jump-and-link return address (OldPC + 4) and update the PC with the pre-computed target.
BRANCH	4'b1010	Evaluate the branch condition (RegisterA - RegisterB) and conditionally update the PC.

[3] Immediate Generator Module

RISC-V encodes immediate constants (used in addi, lw, sw, beq, jal, etc.) differently depending on the instruction format, scattering the immediate bits across non-contiguous positions of the 32-bit instruction word to keep the register-address fields aligned. The Immediate Generator's job is to reassemble those scattered bits into a single, correctly sign-extended 32-bit value, ready for the ALU to use directly.

Port	Direction	Width	Description
instr	input	32	The full current instruction word.
ImmSrc	input	2	Format selector supplied by the Control Unit.
ImmExt	output (reg)	32	Sign-extended, reassembled immediate value.

Format Table

```
always @(*) begin
    case (ImmSrc)
        // 2'b00: I-Type (e.g., lw, addi, jalr)
        2'b00: ImmExt = {{20{instr[31]}}, instr[31:20]};

        // 2'b01: S-Type (e.g., sw)
        2'b01: ImmExt = {{20{instr[31]}}, instr[31:25], instr[11:7]};

        // 2'b10: B-Type (e.g., beq, bne)
        2'b10: ImmExt = {{20{instr[31]}}, instr[7], instr[30:25], instr[11:8], 1'b0};

        // 2'b11: J-Type (e.g., jal)
        2'b11: ImmExt = {{12{instr[31]}}, instr[19:12], instr[20], instr[30:21], 1'b0};

        default: ImmExt = 32'bx;
    endcase
end
```

[4] Memory Module

This module models a single unified memory shared by both instructions and data a design choice typical of textbook multi-cycle RISC-V processors (following the Harris & Harris style architecture). Because instructions and data share one physical memory and one address/data port, the Control Unit must time-multiplex accesses to it across different FSM states (FETCH reads an instruction, MEMREAD reads data, MEMWRITE writes data), which is one of the core reasons the multi-cycle design needs more than one clock cycle per instruction.

Port	Direction	Width	Description
clk	input	1	System clock.
rst	input	1	Active-low asynchronous reset; clears all memory to zero.
WE	input	1	Write Enable.
A	input	32	Byte address; only bits [7:2] are used to index the array.
WD	input	32	Write Data.
RD	output	32	Read Data (combinational read).

[5] MUX_2x1 Module

A generic, reusable 2-to-1 multiplexer. It selects between two same-width buses based on a single select bit. It is deliberately written as a parameterized module so it can be instantiated at any bus width without rewriting the logic.

Port	Direction	Width	Description
WIDTH	parameter	-	Bus width; defaults to 32 bits.
d0	input	WIDTH	Data input selected when s = 0.
d1	input	WIDTH	Data input selected when s = 1.
s	input	1	Select line.
y	output	WIDTH	Selected output.

[6] MUX_3x1 Module

A generic, reusable 3-to-1 multiplexer, again parameterized on bus width. It is used everywhere the datapath must choose among three possible sources for a value, which happens repeatedly in a multi-cycle design because each functional block (the ALU, the result bus) is time-shared across several different logical purposes.

Port	Direction	Width	Description
WIDTH	parameter	-	Bus width; defaults to 32 bits.
d0	input	WIDTH	Data input selected when s = 2'b00.
d1	input	WIDTH	Data input selected when s = 2'b01.
d2	input	WIDTH	Data input selected when s = 2'b10.
s	input	2	2-bit select line.
y	output (reg)	WIDTH	Selected output.

[7] Register File Module

Implements the 32 general-purpose, 32-bit-wide architectural registers (x0-x31) defined by the RISC-V ISA. It supports two simultaneous, purely combinational reads (needed because most instructions reference two source registers, rs1 and rs2) and one clocked write (needed because only one instruction retires per writeback cycle in this design).

Port	Direction	Width	Description
clk	input	1	System clock.
rst	input	1	Active-low asynchronous reset.
reg_write_addr	input	5	Destination register index, instruction[11:7] (rd).
reg_read_addr1	input	5	First source register index, instruction[19:15] (rs1).
reg_read_addr2	input	5	Second source register index, instruction[24:20] (rs2).
reg_write_data	input	32	Value to be written into reg_write_addr.
reg_write_en	input	1	Write enable.
reg_read_data1	output	32	Value currently stored at rs1.
reg_read_data2	output	32	Value currently stored at rs2.

[8] Top-Level Module — `riscv_multi_cycle`

This module has no ports beyond `clk` and `rst`; its entire job is structural it instantiates every submodule described above, declares the wires that connect them, and adds the small number of non-architectural pipeline registers (`PC`, `OldPC`, `Instr`, `Data`, `A`, `B`, `ALUOut`) that are unique to the multi-cycle datapath. Everything about instruction execution flows from how this module wires the Control Unit's outputs to the multiplexers, ALU, memory, and register file.

Non-Architectural Registers

Besides the 32 architectural registers inside `Register_Files`, the multi-cycle datapath needs several 'invisible' pipeline registers that hold intermediate values between cycles. These are declared and clocked directly inside the top-level module rather than as separate submodules:

Register	Updated When	Purpose
<code>pc_reg (PC)</code>	<code>PCWrite</code> asserted	Holds the current program counter value.
<code>OldPC</code>	<code>IRWrite</code> asserted	Snapshots <code>PC</code> at fetch time, used later for <code>OldPC+Imm</code> (branch/jump target) and <code>OldPC+4</code> (jal link value).
<code>Instr_reg (Instr)</code>	<code>IRWrite</code> asserted	Latches the fetched instruction word so it remains stable for the several cycles it takes to execute.
<code>Data_reg (Data)</code>	every cycle	Latches whatever the memory port most recently read; used as the load-data source in <code>MEMWB</code> .
<code>A_reg / B_reg (A, B)</code>	every cycle	Latch the two register-file read outputs (<code>RD1</code> , <code>RD2</code>) so their values remain stable across the several cycles an instruction takes.
<code>ALUOut_reg (ALUOut)</code>	every cycle	Latches the ALU's result from the previous cycle — used both as a computed address and as a computed final result.

Program Counter Logic

The PC is implemented directly in this module as `pc_reg`, updated on the rising clock edge only when `PCWrite` is asserted, from a source called `pc_in_mux` which is simply wired to `Result` (the output of the `result_mux`). Because `Result` can carry `PC+4` (FETCH), the jump target (JAL/via `ALUOut`), or the branch target (BRANCH/via `ALUOut`), routing every possible next-PC value through the same shared `Result` bus is what allows one PC register and one `PCWrite` enable to serve fetch-increment, branches, and jumps alike.

3.5 Verification

[9] Testbench — `riscv_multi_cycle_tb`

The `Riscv_Multi_Cycle_TB` is a self-checking Verilog testbench that instantiates the complete multi-cycle RISC-V processor core, preloads a small hand-written program directly into a unified instruction/data memory, runs it for a sufficient multi-cycle runtime, and automatically compares the resulting register and memory contents against expected golden values.

Structure

Generates a free-running clock with a 10 ns period (5 ns high, 5 ns low).

Defines three reusable checker tasks

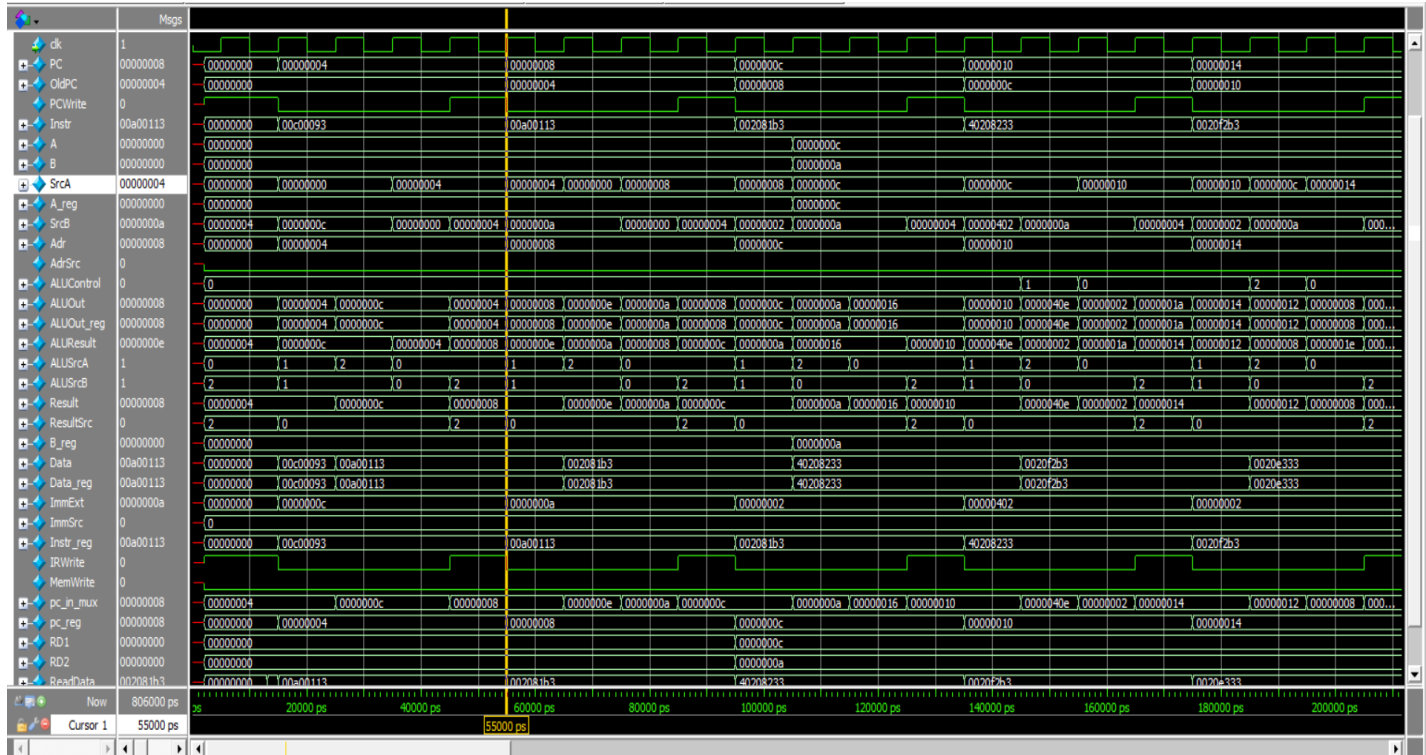
- `check_reg`,
- `check_mem`,
- `check_val`

Test Program

- The **16-instruction** program packed into addresses 0–60 is deliberately built to exercise every opcode while staying inside the instruction memory's addressable window

Addr (Bytes)	Mem Index	Instruction	Hex Encoding	Multi-Cycle Effect	Expected Cycles
0	mem[0]	addi x1, x0, 12	0x00c00093	x1 = 12	4 cycles
4	mem[1]	addi x2, x0, 10	0x00a00113	x2 = 10	4 cycles
8	mem[2]	add x3, x1, x2	0x002081b3	x3 = 22	4 cycles
12	mem[3]	sub x4, x1, x2	0x40208233	x4 = 2	4 cycles
16	mem[4]	and x5, x1, x2	0x0020f2b3	x5 = 8	4 cycles
20	mem[5]	or x6, x1, x2	0x0020e333	x6 = 14	4 cycles
24	mem[6]	slt x7, x2, x1	0x001123b3	x7 = 1 (10 < 12)	4 cycles
28	mem[7]	sw x3, 0(x0)	0x00302023	mem[0] = 22	4 cycles
32	mem[8]	lw x8, 0(x0)	0x00002403	x8 = 22	5 cycles
36	mem[9]	beq x1, x1, 8	0x00108463	Taken: Branch to target PC 44 (mem[11])	3 cycles
40	mem[10]	addi x9, x0, 99	0x06300493	Skipped (x9 remains 0)	—
44	mem[11]	addi x10, x0, 20	0x01400513	Branch target: x10 = 20	4 cycles
48	mem[12]	beq x1, x2, 8	0x00208463	Not Taken (12 ≠ 10): Fall through	3 cycles
52	mem[13]	addi x11, x0, 33	0x02100593	x11 = 33	4 cycles
56	mem[14]	jal x12, 4	0x0040066f	x12 = 60 (Link reg = PC + 4); Jump to 60	3 cycles
60	mem[15]	addi x13, x0, 55	0x03700693	JAL target: x13 = 55	4 cycles

We Simulate the design with modelsim and check the flow with the transcript and the wave forms



```
# ----- Multi-Cycle Check Results -----
# [PASS] 806000 ns : addi x1,x0,12 (x1) = 0x0000000c
# [PASS] 806000 ns : addi x2,x0,10 (x2) = 0x0000000a
# [PASS] 806000 ns : add x3,x1,x2 (x3) = 0x00000016
# [PASS] 806000 ns : sub x4,x1,x2 (x4) = 0x00000002
# [PASS] 806000 ns : and x5,x1,x2 (x5) = 0x00000008
# [PASS] 806000 ns : or x6,x1,x2 (x6) = 0x0000000e
# [PASS] 806000 ns : slt x7,x2,x1 (x7) = 0x00000001
# [PASS] 806000 ns : sw x3,0(x0) (mem[0]) = 0x00000016
# [PASS] 806000 ns : lw x8,0(x0) (x8) = 0x00000016
# [PASS] 806000 ns : addi x9 (skipped by beq) (x9) = 0x00000000
# [PASS] 806000 ns : addi x10 (branch target) (x10) = 0x00000014
# [PASS] 806000 ns : addi x11 (beq not taken) (x11) = 0x00000021
# [PASS] 806000 ns : jal x12 link = PC+4 (x12) = 0x0000003c
# [PASS] 806000 ns : addi x13 (jal target) (x13) = 0x00000037
#
# =====
# ALL 14 CHECKS PASSED
# =====
```

Let's discuss how the core performs these 3 instructions Step-by-Step, Cycle-by-Cycle Execution of the First 3 Instructions

The behavior below was verified by re-implementing the exact register-transfer semantics of the provided Verilog in a cycle-accurate model and simulating it; the resulting register values (x1=12, x2=10, x3=22) match the golden values expected by riscv_multi_cycle_tb.v.

Control Signal Legend

Signal	Meaning
PCWrite	1 = load a new value into the Program Counter this cycle
AdrSrc	0 = memory address comes from PC, 1 = memory address comes from Result (ALUOut/ALUResult)
IRWrite	1 = latch memory read data into the Instruction Register (Instr_reg)
MemWrite	1 = write register B into the unified memory at address Adr
RegWrite	1 = write Result into the register file at address rd
ResultSrc	00 = ALUOut, 01 = Data (memory read data), 10 = ALUResult (fed straight from the ALU output)
ALUSrcA	00 = PC, 01 = OldPC, 10 = register A (A_reg)
ALUSrcB	00 = register B (B_reg), 01 = ImmExt (sign-extended immediate), 10 = constant 4
ALUControl	000 = ADD, 001 = SUB, 010 = AND, 011 = OR, 101 = SLT
ImmSrc	00 = I-type, 01 = S-type, 10 = B-type, 11 = J-type immediate format

The First Three Preloaded Instructions

The testbench preloads the unified memory starting at word address 0. The first three words form a simple arithmetic sequence: load two immediate values into x1 and x2, then add them into x3.

Addr	Hex	Assembly	opcode	rd	funct3	rs1	rs2/imm[11:5]	imm[11:0] / funct7
0	0x00c00093	addi x1, x0, 12	0010011	00001 (x1)	000	00000 (x0)	-	imm = 000000001100 = 12
4	0x00a00113	addi x2, x0, 10	0010011	00010 (x2)	000	00000 (x0)	-	imm = 000000001010 = 10
8	0x002081b3	add x3, x1, x2	0110011	00011 (x3)	000	00001 (x1)	00010 (x2)	funct7 = 00000000

Instruction 1: addi x1, x0, 12

Encoding 0x00C00093.

Adds the immediate 12 to x0 (always zero) and stores the result in x1.

Executes over cycles 1-4, while PC advances from 0 to 4.

Cycle 1 — State: **FETCH**

Control signals: AdrSrc=0 , IRWrite=1 , ALUSrcA=00(PC) , ALUSrcB=10(4), ALUOp=ADD , ResultSrc=10(ALUResult) , PCWrite=1

Adr = PC = 0x00000000 , so the memory is addressed at word 0.

ReadData = mem[0] = 0x00C00093 (the addi x1,x0,12 instruction).

ALU computes SrcA (PC=0) + SrcB (constant 4) = ALUResult = 4.

This is the classic PC+4 calculation, and because ResultSrc=10 selects ALUResult, Result = 4 as well.

At the rising clock edge that ends this cycle: Instr_reg <= 0x00C00093 (because IRWrite=1), OldPC <= PC (=0), Data_reg <= ReadData (=0x00C00093, latched

every cycle regardless of use), $pc_reg \leq Result (=4, \text{ because } PCWrite=1)$ so PC becomes 4, and $ALUOut_reg \leq ALUResult (=4)$.

A_reg and B_reg also latch RD1/RD2 from the register file every cycle (unconditionally in this design); since Instr is still 0 from reset, RD1=RD2=0 here, which is harmless because this instruction does not need A/B yet.

Cycle 2 — State: **DECODE**

Control signals: ALUSrcA=01(OldPC), ALUSrcB=01(ImmExt), ALUOp=ADD, ImmSrc=00 (I-type, because op=0010011)

Instr is now 0x00C00093, so op[6:0]=0010011 selects the I-type immediate format: ImmExt = sign-extend(instr[31:20]) = 12.

The controller also pre-computes a potential branch/jump target here (a feature of this FSM shared by every instruction): SrcA = OldPC (0) and SrcB = ImmExt (12), so ALUResult = 0 + 12 = 12. This value is latched into ALUOut_reg at the end of the cycle, but for an addi instruction it is simply unused later.

RD1 and RD2 are read from the register file using rs1=x0 and the addi encoding's bit field that would be rs2 (irrelevant for I-type) — since x0 always reads 0, this is harmless.

At the clock edge: $ALUOut_reg \leq 12$, $A_reg \leq RD1(=0)$, $B_reg \leq RD2(=0)$, $Data_reg \leq ReadData$ (memory word 1, unused). PC, OldPC and Instr_reg do not change (IRWrite=0, PCWrite=0).

FSM moves to EXECUTEI next (because op=0010011 is the I-type ALU opcode).

Cycle 3 — State: **EXECUTEI**

Control signals: ALUSrcA=10(A), ALUSrcB=01(ImmExt), ALUOp=10 → ALU decoder looks at funct3=000, {op[5],funct7_5} → ADD

Now the real computation happens: SrcA = A_reg = 0 (value of x0), SrcB = ImmExt = 12.

funct3 = 000 and op[5]=0 means the ALU decoder selects ADD

(ALUControl=000), so ALUResult = 0 + 12 = 12. This is the addi result: $x1 \leftarrow x0 + 12$.

At the clock edge, $ALUOut_reg \leq 12$ (the result is captured for the write-back stage). No register or PC write happens yet (RegWrite=0, PCWrite=0).

Cycle 4 — State: **ALUWB**

Control signals: ResultSrc=00(ALUOut), RegWrite=1

Result = ALUOut_reg = 12 (the value captured in the previous cycle).

Because RegWrite=1, the register file writes reg_file[rd] where rd = Instr[11:7] = 00001 (x1). So $x1 \leftarrow 12$.

This completes the addi x1,x0,12 instruction after exactly 4 clock cycles (FETCH → DECODE → EXECUTEI → ALUWB). The FSM returns to FETCH for the next instruction, with PC already sitting at 4 (updated back in cycle 1).

Instruction 2: addi x2, x0, 10

Encoding 0x00A00113. Adds the immediate 10 to x0 and stores the result in x2. Executes over cycles 5-8, while PC advances from 4 to 8.

Cycle 5 — State: **FETCH**

Control signals: Same FETCH signals as before: IRWrite=1, ALUSrcA=00(PC), ALUSrcB=10(4), ResultSrc=10, PCWrite=1

Adr = PC = 4, so ReadData = mem[1] = 0x00A00113 (addi x2,x0,10).

ALUResult = PC(4) + 4 = 8. Because ResultSrc=10, Result = 8.

At the clock edge: Instr_reg ≤ 0x00A00113, OldPC ≤ 4, pc_reg ≤ 8 (PC becomes 8), ALUOut_reg ≤ 8, Data_reg ≤ ReadData.

Cycle 6 — State: **DECODE**

Control signals: ALUSrcA=01(OldPC), ALUSrcB=01(ImmExt), ALUOp=ADD, ImmSrc=00

Instr is now 0x00A00113. op=0010011 → ImmSrc=00 (I-type) → ImmExt = sign-extend(instr[31:20]) = 10.

SrcA = OldPC = 4, SrcB = ImmExt = 10, so ALUResult = 14 (an unused speculative target address, same as before).

RD1/RD2 are read using rs1=x0 (→ 0); A_reg and B_reg latch 0 at the edge.

ALUOut_reg ≤ 14.

Cycle 7 — State: **EXECUTEI**

Control signals: ALUSrcA=10(A), ALUSrcB=01(ImmExt), ALUOp→ADD
SrcA = A_reg = 0 (x0), SrcB = ImmExt = 10. ALUControl decodes to ADD (funct3=000, op[5]=0).

ALUResult = 0 + 10 = 10 — this is the addi result for x2.

At the clock edge, ALUOut_reg ≤ 10.

Cycle 8 — State: **ALUWB**

Control signals: ResultSrc=00(ALUOut), RegWrite=1

Result = ALUOut_reg = 10.

rd = Instr[11:7] = 00010 (x2), RegWrite=1 → x2 ← 10.

Instruction 2 (addi x2,x0,10) completes after 4 cycles; PC is already 8, ready to fetch instruction 3.

Instruction 3: add x3, x1, x2

Encoding 0x002081B3. R-type instruction: x3 ≤ x1 + x2. Executes over cycles 9-12, while PC advances from 8 to 12. This is the first instruction to actually consume values produced by earlier instructions (x1 and x2), exercising the register file read/write path fully.

Cycle 9 — State: **FETCH**

Control signals: IRWrite=1, ALUSrcA=00(PC), ALUSrcB=10(4), ResultSrc=10, PCWrite=1

Adr = PC = 8, so ReadData = mem[2] = 0x002081B3 (add x3,x1,x2).

ALUResult = PC(8) + 4 = 12 = Result. At the clock edge: Instr_reg ≤ 0x002081B3, OldPC ≤ 8, pc_reg ≤ 12, ALUOut_reg ≤ 12.

Cycle 10 — State: **DECODE**

Control signals: ALUSrcA=01(OldPC), ALUSrcB=01(ImmExt), ALUOp=ADD

Instr is now 0x002081B3. Its opcode 0110011 is R-type, so the immediate generator output is not architecturally meaningful here, but the datapath still forms a speculative OldPC+ImmExt value (harmless for R-type).

This is the important cycle for register operands: the register file combinationaly reads $rs1 = \text{Instr}[19:15] = 00001$ (x1) and $rs2 = \text{Instr}[24:20] = 00010$ (x2). Because x1 and x2 were already written back in cycles 4 and 8, $RD1 = 12$ and $RD2 = 10$ right now.

At the rising edge ending this cycle, $A_reg \leq RD1 = 12$ and $B_reg \leq RD2 = 10$ — these are the operands the ALU will use next cycle. $ALUOut_reg$ also latches the speculative $\text{OldPC} + \text{Imm}$ result ($8 + 2 = 10$, unused).

Next state is EXECUTER because $op = 0110011$ (R-type).

Cycle 11 — State: **EXECUTER**

Control signals: $ALUSrcA = 10(A)$, $ALUSrcB = 00(B)$, $ALUOp = 10 \rightarrow \text{funct3} = 000$, $\{op[5], \text{funct7_5}\} = 10 \rightarrow \text{ADD}$

$SrcA = A_reg = 12$ (x1), $SrcB = B_reg = 10$ (x2).

$\text{funct3} = 000$ and $\{op[5] = 1, \text{funct7_5} = 0\} \neq 11$, so the ALU decoder selects ADD, $ALUControl = 000$.

$ALUResult = 12 + 10 = 22$.

At the clock edge, $ALUOut_reg \leq 22$ (result captured for write-back).

Cycle 12 — State: **ALUWB**

Control signals: $\text{ResultSrc} = 00(\text{ALUOut})$, $\text{RegWrite} = 1$

$\text{Result} = ALUOut_reg = 22$.

$rd = \text{Instr}[11:7] = 00011$ (x3), $\text{RegWrite} = 1 \rightarrow x3 \leftarrow 22$.

$\text{add } x3, x1, x2$ completes after 4 cycles, giving the expected value $x3 = x1 + x2 = 12 + 10 = 22$, which matches the testbench's expected $\text{check_reg}(3, 22, \dots)$.

Cycle-by-Cycle Summary Table

Cycle	State	Instruction	PC (before)	Key Action	Register Written
1	FETCH	<code>addi x1,x0,12</code>	0	Fetch instr, $PC \leq 4$	-
2	DECODE	<code>addi x1,x0,12</code>	4	Decode, $\text{ImmExt} = 12$	-
3	EXECUTEI	<code>addi x1,x0,12</code>	4	ALU: $0 + 12 = 12$	-

Cycle	State	Instruction	PC (before)	Key Action	Register Written
4	ALUWB	addi x1,x0,12	4	Writeback	x1 = 12
5	FETCH	addi x2,x0,10	4	Fetch instr, PC<=8	-
6	DECODE	addi x2,x0,10	8	Decode, ImmExt=10	-
7	EXECUTEI	addi x2,x0,10	8	ALU: 0+10=10	-
8	ALUWB	addi x2,x0,10	8	Writeback	x2 = 10
9	FETCH	add x3,x1,x2	8	Fetch instr, PC<=12	-
10	DECODE	add x3,x1,x2	12	Read RF: A<=x1=12, B<=x2=10	-
11	EXECUTER	add x3,x1,x2	12	ALU: 12+10=22	-
12	ALUWB	add x3,x1,x2	12	Writeback	x3 = 22

Architectural State After the First 3 Instructions

After cycle 12 (end of instruction 3), the visible architectural state is:

PC = 12 (0x0000000C) — points to instruction 4 (sw x3,0(x0)).

x1 = 12 x2 = 10

x3 = 22 (= x1 + x2, verified against the testbench's expected check_reg(3, 22, ...))

Memory is unchanged (no MemWrite has occurred yet).

Key Observations About This Multi-Cycle Design

Every instruction spends exactly 1 cycle in FETCH and 1 cycle in DECODE before branching to an instruction-specific path; addi/add instructions then need exactly 2 more cycles (EXECUTEI or EXECUTER, then ALUWB), for a fixed total of 4 cycles.

The DECODE state always speculatively computes $\text{OldPC} + \text{ImmExt}$ into `ALUOut_reg`, even for instructions (like `addi/add`) that never use that value — this keeps the control logic uniform across all instruction types (it matters for `beq/jal`, which reuse this pre-computed target).

`A_reg` and `B_reg` are latched from the register file on every clock edge (not just during DECODE). This is harmless because, by the time `EXECUTER/EXECUTEI` needs A and B, the most recent DECODE cycle has already re-latched the correct operand values from the current instruction's `rs1/rs2` fields.

Because the register file write (`ALUWB, RegWrite=1`) for one instruction happens two cycles before the next instruction reaches its own DECODE stage, forwarding is not required: by the time `add x3,x1,x2` reads `x1` and `x2` in cycle 10, both values were already committed to the register file (in cycles 4 and 8).

4. Conclusion

This technical report successfully demonstrates the design, Verilog implementation, and functional verification of both Single-Cycle and Multi-Cycle 32-bit RISC-V (RV32I subset) processors.

Single-Cycle Design: Executes every instruction in a single long clock period by utilizing separate instruction and data memories alongside dedicated hardware adders. While it offers simple control logic without hazards, it suffers from a longer critical path and lower clock frequency.

Multi-Cycle Design: Improves resource efficiency by time-sharing hardware components (such as a single unified memory and ALU) across a 12-state Finite State Machine (FSM). By breaking execution into shorter clock cycles (taking 3 to 5 cycles per instruction), it reduces hardware overhead and improves maximum operational clock frequency.

Full cycle-accurate ModelSim simulations confirmed 100% test pass rates across all supported instruction formats (R, I, S, B, J), validating correct execution flow, branch/jump handling, and register/memory state transitions.

5. ABP bus and Address decoder

The APB_bus module is the core finite state machine (FSM) that orchestrates the APB transactions, handling address, data, and control signal latching, as well as core stalling during peripheral access.

5.1 Ports Description

Port Name	Direction	Width	Description
PCLK	Input	1-bit	APB system clock.
PRST_n	Input	1-bit	Active-low peripheral reset.
uart_sel	Input	1-bit	Selection signal for the UART peripheral.
i2c_sel	Input	1-bit	Selection signal for the I2C peripheral.
addr_fault	Input	1-bit	Indicates an invalid core address request.
core_addr	Input	32-bit	Memory address requested by the core.
PREADY_uart	Input	1-bit	Ready signal from the UART peripheral.
PREADY_i2c	Input	1-bit	Ready signal from the I2C peripheral.

core_memread	Input	1-bit	Core read enable signal.
core_memwrite	Input	1-bit	Core write enable signal.
PRDATA_UART	Input	32-bit	Read data from the UART peripheral.
PRDATA_I2C	Input	32-bit	Read data from the I2C peripheral.
core_wdata	Input	32-bit	Data provided by the core to be written.
PSEL_uart	Output (reg)	1-bit	Registered APB select signal for UART (PSELx).
PSEL_i2c	Output (reg)	1-bit	Registered APB select signal for I2C (PSELx).
PEN	Output (reg)	1-bit	APB enable signal (PENABLE).
PWRITE	Output (reg)	1-bit	APB write control signal (PWRITE).
PADDR	Output (reg)	32-bit	Registered APB address (PADDR).

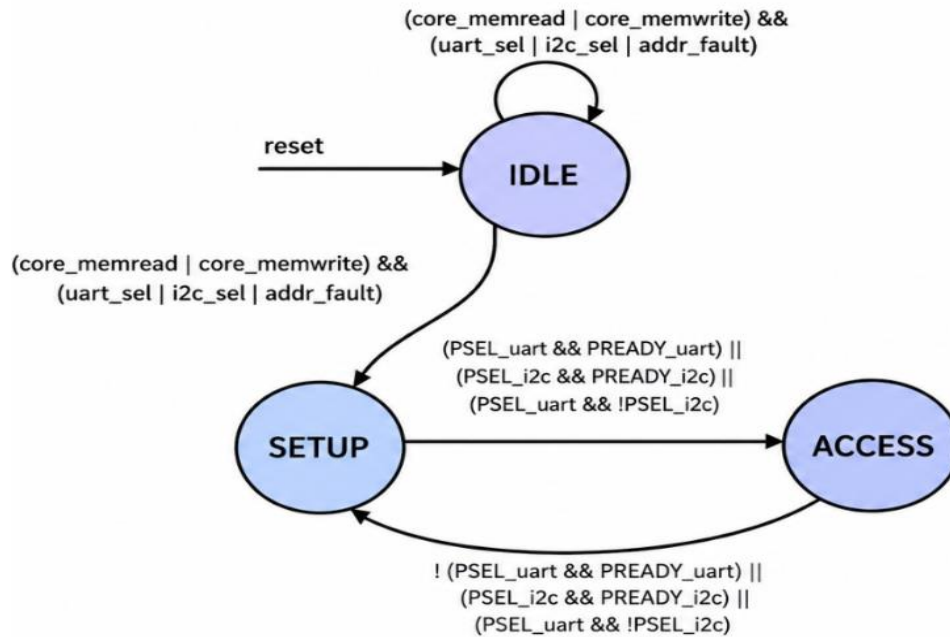
PWDATA	Output (reg)	32-bit	Registered APB write data (PWDATA).
core_stall	Output	1-bit	Stalls the processor core during an active APB transaction.
core_rdata	Output (reg)	32-bit	Data returned to the core after a read transaction.

5.2 FSM State Machine

The module utilizes a 3-state Moore/Mealy machine to execute APB transfers. State transitions occur on the positive edge of PCLK, with asynchronous reset via PRST_n.

- **IDLE (2'b00):** The default resting state. The bus remains here until a core memory access (core_memread or core_memwrite) occurs alongside a valid target (uart_sel, i2c_sel, or addr_fault). If triggered, the inputs (core_addr, core_wdata, core_memwrite) are latched into APB registers (PADDR, PWDATA, PWRITE, PSEL_x), and the state moves to SETUP.
- **SETUP (2'b01):** Marks the setup phase of the APB transaction. The PEN (PENABLE) signal is asserted high to begin the access phase on the next clock edge. The state transitions to ACCESS unconditionally.

- **ACCESS (2'b10):** The access phase. The FSM waits for the selected peripheral to assert its respective PREADY signal. If PREADY is high (or if an address fault occurred, represented by no PSEL being active), the state returns to IDLE. PEN and PSEL_x are de-asserted. During a read operation, PRDATA from the active peripheral is latched into core_rdata (or 0 in case of a fault). If PREADY is low, the FSM remains in ACCESS, simulating wait states.



5.3 Data Flow & Logic

Stall Logic: The core_stall signal is actively asserted high whenever a transaction is initiated, during the SETUP phase, and throughout the ACCESS phase until the target peripheral asserts PREADY. This ensures the core does not overwrite data or addresses before the APB transaction completes.

5.4 Test Cases Executed

The verification testbench for the APB bus, simulating a 100MHz clock (10ns period) and validating the FSM against various transaction types and corner cases.

1. **Reset Flow:** Initializes the system with a hardware reset (PRST_n = 0 then 1).

2. UART Write Transaction (No Wait States): Simulates a core writing 0xAA55CC33 to UART at 0x40001000. Validates a zero-wait-state peripheral where PREADY_uart is asserted immediately in ACCESS.

3. I2C Read Transaction (With Wait States): Simulates a core reading from I2C at 0x40002000. Demonstrates wait state handling by holding PREADY_i2c low for two extra cycles before providing 0xDEADBEEF. Verifies that core_rdata latches the read data properly upon readiness.

4. Address Fault Handling: Simulates an access to an unmapped address (0xFFFFFFFF). Validates that the FSM transitions gracefully through SETUP and ACCESS, and exits without a PREADY signal since no actual peripheral is selected, preventing the bus from deadlocking.

6. Address decoder

6.1 Ports Description

Port Name	Direction	Width	Description
core_addr_decoder	Input	32-bit	Memory address requested by the core.
access_req	Input	1-bit	High when the core requests an access (Read or Write).
uart_sel	Output	1-bit	High when address maps to UART base.
i2c_sel	Output	1-bit	High when address maps to I2C base.
addr_fault	Output	1-bit	High when access_req is active but the

			address maps to neither.
core_addr	Output	32-bit	Pass-through latching of the core_addr_decoder.

6.2 Internals & Flow

Memory Map Configuration:

- UART_BASE_24: 0x400000 (Maps to 32-bit range 0x40000000 to 0x400000FF)
- I2C_BASE_24: 0x400001 (Maps to 32-bit range 0x40000100 to 0x400001FF)

The decoder analyzes the upper 24 bits (bits [31:8]) of the incoming core address. If it matches the UART or I2C base during a valid access request, the respective select line is asserted. If an access is requested to an unmapped region, `addr_fault` is flagged to trigger the bus's fault handling.

7. I2C Master Peripheral

7.1 System Overview

This section documents the I2C master peripheral built for the RISC-V SoC Core. I2C (Inter-Integrated Circuit) is a two-wire serial protocol: SCL carries a clock driven by the master, and SDA carries data shared between master and slave. A transaction always follows the same shape: the master issues a START condition, sends a 7-bit slave address plus a read/write bit, the addressed slave pulls SDA low for one clock cycle to ACKnowledge, one or more 8-bit data bytes are then transferred one bit per SCL pulse (each byte followed by its own ACK/NACK bit), and the master finally issues a STOP condition to release the bus.

In this SoC, the I2C master sits on the peripheral side of the AXI4 interconnect: the RISC-V core talks to AXI4, the AXI-to-APB bridge converts that into an APB transaction, and this module's `apb_interface` answers as an APB slave. Software

configures the peripheral and starts transactions purely through memory-mapped registers exposed by the `register_file` — it writes the desired SCL divider, the byte to send, the read/write direction, and a start pulse, then lets the hardware FSM run the actual bit-level protocol autonomously.

The peripheral is built from six source files, each verified as an independent building block before being wired together in the top-level module, plus a sanity testbench:

clock_generator.v : generates the SCL clock line from the system clock, at a rate set at run time by the `div_value` register.

shift_reg.v : serializes an outgoing byte onto SDA one bit per SCL pulse (TX) or deserializes an incoming byte off SDA (RX).

i2c_fsm.v : the 9-state controller that sequences an entire transaction — START, ADDRESS, ACK, DATA (TX or RX), ACK, STOP — and drives every control signal in the datapath.

register_file.v : the software-visible register map: SCL divider, TX/RX data, start pulse, read/write mode, and status.

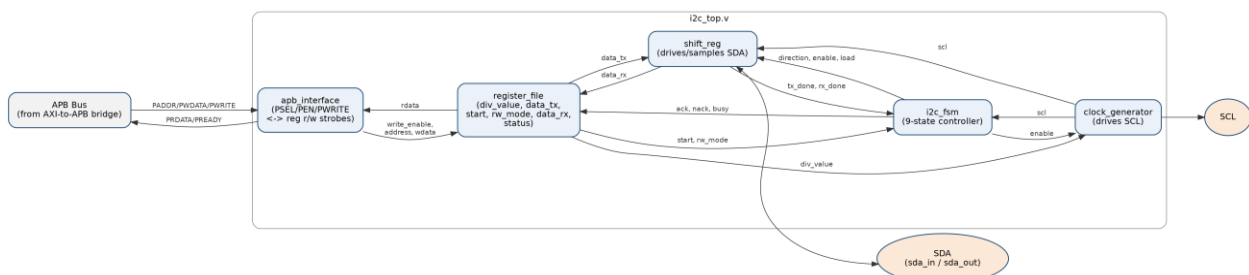
apb_interface.v : the APB slave port that turns PSEL/PEN/PWRITE bus transfers into simple read/write strobes for the register file.

i2c_top.v : top-level module that instantiates and wires all of the above together into one APB peripheral.

i2c_top_sanity_tb.v : a self-checking testbench that drives the whole peripheral through its APB port.

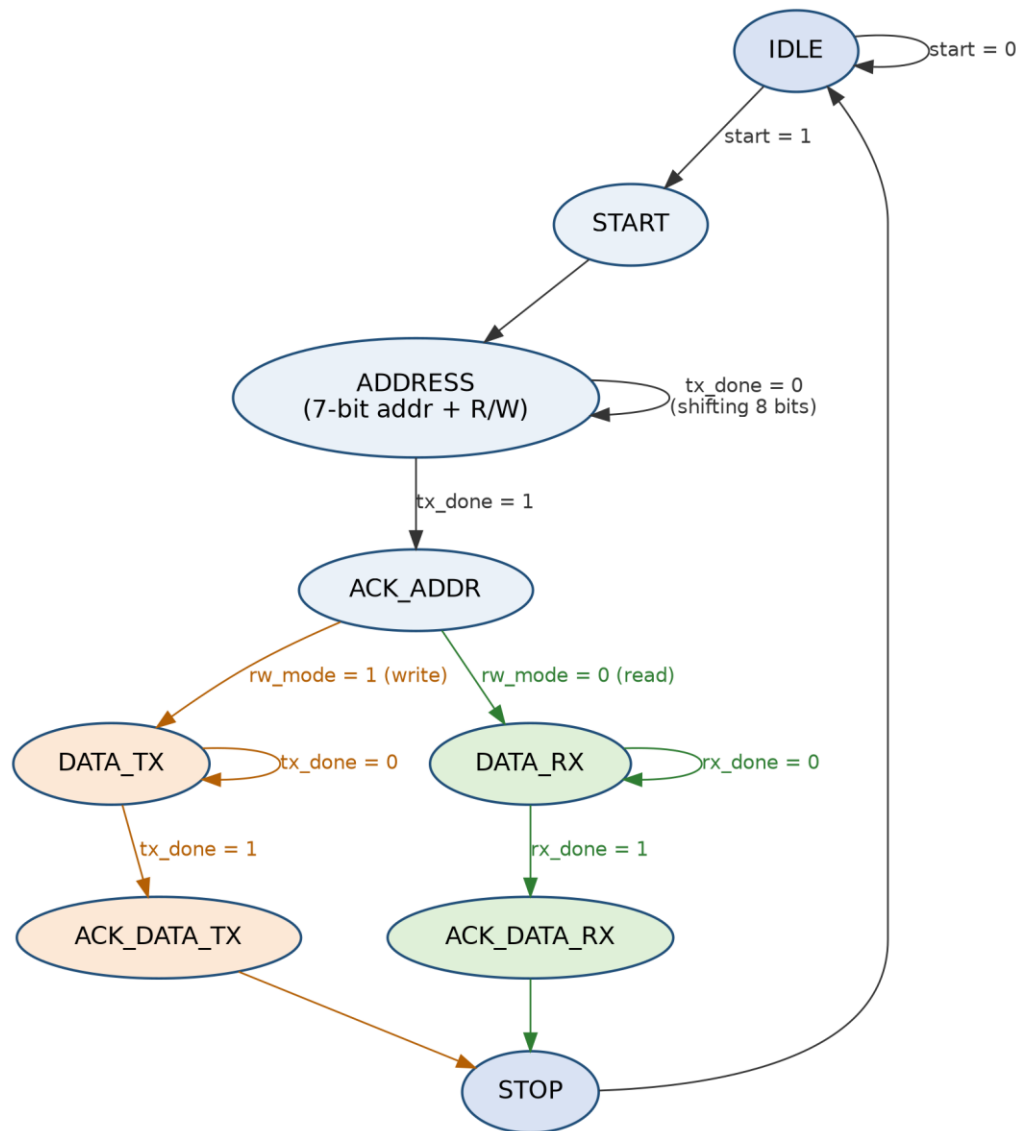
7.2 Implementation

Block Diagram for the Whole Design



apb_interface decodes every APB transfer into a register_file read or write. register_file is the only module software ever touches: it holds div_value (the SCL clock divider), data_tx (the byte queued for transmission), rw_mode (write or read transaction), the start pulse, data_rx (the last byte received) and a status bit made of busy/ack/nack. Writing start hands control to i2c_fsm, which drives clock_generator (enable, producing SCL) and shift_reg (direction, enable and load, producing/sampling SDA) for the whole transaction, then reports tx_done/rx_done back to the FSM and busy/ack/nack back to register_file so software can poll completion.

I2C Master Control FSM — TX and RX Paths



The FSM is a 9-state Moore machine. IDLE, START, ADDRESS and ACK_ADDR are shared by every transaction: the address phase always transmits, since the 7-bit slave address plus R/W bit is sent by the master regardless of whether the transaction itself is a read or a write. The path only forks once, at ACK_ADDR, based on rw_mode: a write transaction takes the DATA_TX / ACK_DATA_TX branch, shifting the queued byte out through shift_reg one bit per SCL pulse until tx_done; a read transaction takes the DATA_RX / ACK_DATA_RX branch, shifting bits in from SDA until rx_done. Both branches rejoin at STOP, which releases the bus and returns the FSM to IDLE.

The FSM's state register is clocked on sys_clk rather than scl. SCL itself only toggles while clock_generator's enable input is high, and enable is one of the FSM's own outputs — so if the state register were clocked on scl, the FSM could never leave IDLE (it needs an SCL edge to start toggling, but SCL needs the FSM to have already started to toggle at all). Running the state register on the free-running system clock breaks that deadlock. shift_reg is the one piece of logic that must still move exactly one bit per real SCL pulse, so it alone is clocked directly on scl.

load must reach shift_reg as a pulse exactly one SCL cycle wide, since shift_reg samples it on the rising edge of scl. Comparing “previous state $\neq$ current state” in the sys_clk domain only produces a pulse one sys_clk cycle wide — far narrower than one SCL period — so shift_reg could miss it depending on phase. To fix this, i2c_fsm re-samples current_state itself on posedge scl (state_scl_sync). Because current_state is stable for many sys_clk cycles between SCL edges, comparing it against state_scl_sync is true for exactly the first SCL edge after entering ADDRESS or DATA_TX, and false on every SCL edge after that — giving load a clean, one-SCL-cycle pulse.

7.3 Building Blocks

1. Clock Generator (clock_generator.v)

Generates the SCL clock line by toggling scl every time an internal counter reaches half of div_value, giving SCL a period of $(div_value + 1)$ system-clock cycles. Because div_value is a run-time register rather than a synthesis-time parameter, software can change the I2C bus speed without resynthesizing the design.

Port	Dir	Width	Description
sys_clk	input	1	System clock — SCL toggling is derived from this clock.
rst	input	1	Active-low asynchronous reset; clears the counter and forces scl low.
enable	input	1	From i2c_fsm. SCL only toggles while a transaction is in progress.
div_value	input	8	Run-time SCL divider from register_file; sets the bus clock speed.
scl	output	1	The generated I2C clock line, driven onto the SoC's SCL pin.

2. Shift Register (shift_reg.v)

Moves one data byte across SDA, one bit per SCL edge, in either direction. In transmit mode it loads data_in into an internal tx_buffer on a one-shot load pulse, then shifts tx_buffer out onto sda_out MSB-first. In receive mode it shifts bits sampled from sda_in into data_out. Either way, an internal 3-bit bit_count tracks progress through the byte and raises tx_done or rx_done once all 8 bits have moved.

Port	Dir	Width	Description
scl	input	1	Clock — shift_reg moves exactly one bit per rising edge of SCL.
rst	input	1	Active-low asynchronous reset; clears buffers, counters and done flags.

Port	Dir	Width	Description
data_in	input	8	Byte to transmit, loaded into tx_buffer when load is asserted.
sda_in	input	1	SDA line sampled bit-by-bit while receiving (direction = 0).
direction	input	1	1 = transmit (TX) mode, 0 = receive (RX) mode.
enable	input	1	From i2c_fsm; shifting/sampling only happens while enable = 1.
load	input	1	One-shot pulse from i2c_fsm that parallel-loads data_in into tx_buffer.
data_out	output	8	Byte assembled from SDA while receiving.
rx_done	output	1	Pulses high for one SCL cycle once all 8 RX bits have been shifted in.
tx_done	output	1	Pulses high for one SCL cycle once all 8 TX bits have been shifted out.
sda_out	output	1	SDA line driven bit-by-bit while transmitting.

3. FSM Controller (i2c_fsm.v)

The 9-state Moore controller described above (IDLE, START, ADDRESS, ACK_ADDR, DATA_TX, ACK_DATA_TX, DATA_RX, ACK_DATA_RX, STOP), built with the classic three-always-block style: a state register, a combinational next-state block using ternary operators, and a combinational output block that defaults every output to 0 and then overrides per state. It drives

direction/enable/load into shift_reg, enable into clock_generator, and reports ack/nack/busy back to register_file for software to poll.

Two known gaps remain, left in place with explanatory comments in the source rather than fixed silently: nack is currently always driven 0, since no module yet samples SDA during the ACK bit and compares it against the expected level; and data_rx is carried as an FSM input for that future ACK/NACK-compare logic but is not yet used anywhere in the current next-state or output logic.

Port	Dir	Width	Description
sys_clk	input	1	Drives the FSM's state register (see deadlock note above).
scl	input	1	Used only to generate a clean, one-SCL-cycle-wide load pulse.
rst	input	1	Active-low asynchronous reset; returns the FSM to IDLE.
rx_done	input	1	From shift_reg; advances DATA_RX to ACK_DATA_RX.
tx_done	input	1	From shift_reg; advances ADDRESS/DATA_TX to their ACK states.
data_rx	input	8	Received byte from shift_reg; reserved for future ACK/NACK checking.
start	input	1	From register_file; a start pulse moves IDLE to START.
rw_mode	input	1	From register_file (software): 1 = write, 0 = read. Decides the ACK_ADDR fork.

Port	Dir	Width	Description
direction	output	1	To shift_reg: 1 while transmitting (ADDRESS/DATA_TX/ACK_DATA_RX), 0 while receiving.
enable	output	1	To shift_reg and clock_generator; high for every state except IDLE and STOP.
load	output	1	One-shot pulse to shift_reg on the first SCL edge of ADDRESS or DATA_TX.
ack	output	1	Asserted during ACK_DATA_RX (master ACKs a received byte).
nack	output	1	Reserved for NACK signalling; always 0 until ACK/NACK detection is added.
busy	output	1	1 whenever the FSM is anywhere other than IDLE; feeds register_file's status bit.

4. Register File (register_file.v)

The only part of the peripheral software can see. It exposes a small 8-bit-addressed register space behind the APB interface, latching writes from the bus into the hardware-facing outputs below, and muxing hardware-facing inputs back out as read data.

Port	Dir	Width	Description
pclk	input	1	APB clock — register writes occur on its rising edge.

Port	Dir	Width	Description
presetn	input	1	Active-low asynchronous reset; clears every register to 0.
write_enable	input	1	From apb_interface; a valid APB write is in progress.
read_enable	input	1	From apb_interface; a valid APB read is in progress.
address	input	8	Internal register address, from apb_interface (PADDR[7:0]).
wdata	input	8	Write data from apb_interface (PWDATA[7:0]).
rdata	output	8	Read data returned to apb_interface for the current address.
div_value	output	8	To clock_generator — software-configured SCL divider.
data_tx	output	8	To shift_reg — the byte queued for transmission.
start	output	1	To i2c_fsm — a one-shot, self-clearing start pulse.
rw_mode	output	1	To i2c_fsm — 1 = write, 0 = read.
data_rx	input	8	From shift_reg (via i2c_top) — last byte received, readable by software.

Port	Dir	Width	Description
ack	input	1	From i2c_fsm; folded into the status register.
nack	input	1	From i2c_fsm; folded into the status register.
busy	input	1	From i2c_fsm; folded into the status register.

Proposed internal address map (to be confirmed against any shared team convention):

Address	Register	Access	Description
0x00	div_value	R/W	SCL clock divider.
0x01	data_tx	W	Byte to transmit.
0x02	data_rx	R	Last byte received.
0x03	start	W	Self-clearing start pulse (bit 0).
0x04	rw_mode	W	1 = write transaction, 0 = read transaction.
0x05	status	R	{5'b0, busy, nack, ack}.

5. APB Interface (apb_interface.v)

The APB slave port that connects this peripheral to the AXI-to-APB bridge. A transfer is considered valid on the ACCESS phase (PSEL_i2c & PEN); the module then simply forwards PWRITE, the low byte of PADDR/PWDATA, and register_file's rdata in both directions. This slave never inserts wait states — PREADY_i2c is asserted as soon as PSEL_i2c and PEN are both seen — which is

a safe default but assumes the RISC-V core can already tolerate an APB-style single-cycle ready response.

Port	Dir	Width	Description
PCLK	input	1	APB clock, shared with register_file's pclk.
PRST_n	input	1	Active-low APB reset.
PSEL_i2c	input	1	Peripheral select from the AXI-to-APB bridge's address decoder.
PEN	input	1	APB enable; together with PSEL_i2c marks the ACCESS phase.
PWRITE	input	1	1 = write transfer, 0 = read transfer.
PADDR	input	32	APB address bus; only PADDR[7:0] is used as this peripheral's internal address.
PWDATA	input	32	APB write-data bus; only PWDATA[7:0] is used.
PRDATA_I2C	output	32	Read data returned to the bus, zero-extended from register_file's rdata.
PREADY_i2c	output	1	Asserted whenever PSEL_i2c & PEN are both high (no wait states).
write_enable	output	1	To register_file — access & PWRITE.
read_enable	output	1	To register_file — access & ~PWRITE.
address	output	8	To register_file — PADDR[7:0].

Port	Dir	Width	Description
wdata	output	8	To register_file — PWDATA[7:0].
rdata	input	8	From register_file, forwarded onto PRDATA_I2C.

6. I2C Top (i2c_top.v)

The top-level module: instantiates apb_interface, register_file, i2c_fsm, clock_generator and shift_reg, and wires them together exactly as shown in the block diagram above. This is the module the rest of the SoC (the AXI-to-APB bridge) actually instantiates.

Port	Dir	Width	Description
PCLK	input	1	APB clock.
PRST_n	input	1	Active-low APB reset.
PSEL_i2c	input	1	Peripheral select from the AXI-to-APB bridge.
PEN	input	1	APB enable.
PWRITE	input	1	1 = write, 0 = read.
PADDR	input	32	APB address bus.
PWDATA	input	32	APB write-data bus.
PRDATA_I2C	output	32	APB read-data bus.

Port	Dir	Width	Description
PREADY_i2c	output	1	APB ready (always single-cycle).
sda_in	input	1	SDA line sampled by shift_reg while receiving.
sda_out	output	1	SDA line driven by shift_reg while transmitting.

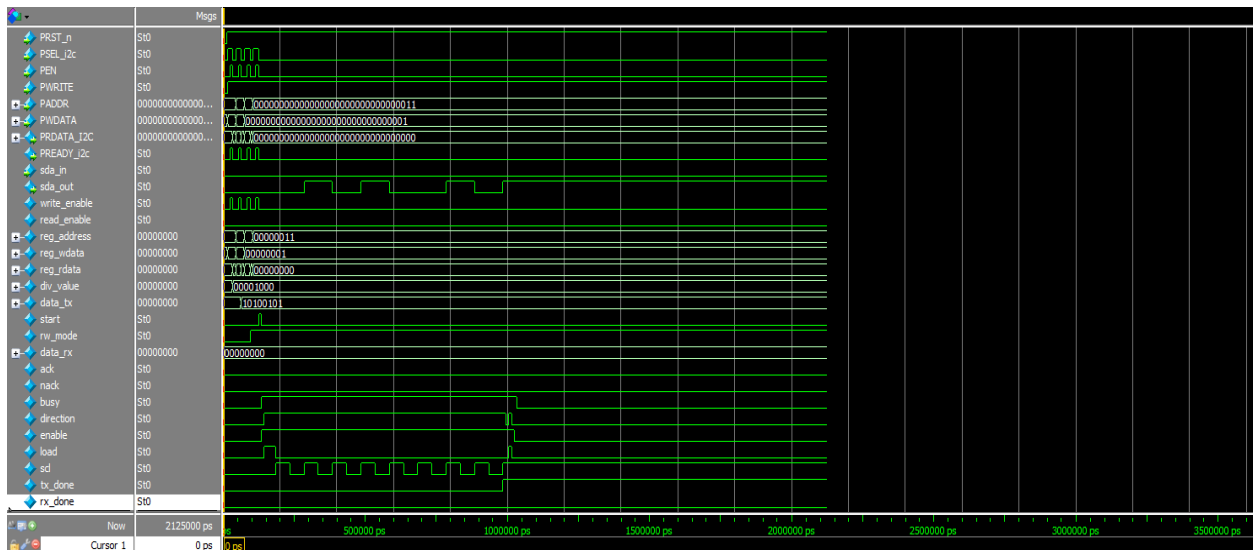
Note: sda_in and sda_out are still two separate wires rather than one true bidirectional SDA line. A tri-state buffer, with its output-enable driven by direction, still needs to be added before this connects to a real, single-wire SDA pin.

7.4 Verification

7. i2c_top_sanity_tb

A sanity-level testbench that instantiates i2c_top and drives it purely through its APB port, the same way the AXI-to-APB bridge eventually will. An apb_write task performs one complete APB write (asserting PSEL_i2c/PWRITE/PADDR/PWDATA, then PEN, then releasing both) so the main initial block can issue readable, one-line register writes.

After releasing reset, the testbench configures the peripheral for a write transaction and pulses start:



Address	Value	Effect
0x00	8'd8	div_value = 8 — fast SCL toggling for simulation speed.
0x01	8'hA5	data_tx = 0xA5 — the byte to be transmitted.
0x04	8'h01	rw_mode = 1 — configure a write transaction.
0x03	8'h01	start = 1 — one-shot pulse that kicks off the FSM.

The simulation then runs for a fixed window and finishes, dumping a VCD waveform (i2c_top_sanity.vcd) so the resulting SCL/SDA activity and internal FSM state can be inspected by eye. This confirms the APB path into register_file and the FSM's start-up sequence; it is a sanity check rather than the self-checking, expected-vs-actual style testbench used for the RISC-V core — a dedicated i2c_fsm testbench with automatic pass/fail checks is still on the to-do list.

8. Future Work

In the future, we plan to further develop and strengthen the system by improving the connection between the RISC-V core and the APB peripherals and by completing the required stall support. The I2C interface will also be improved for real hardware operation, including proper SDA tri-state control and ACK/NACK detection. In addition, the register address map and system clock settings will be finalized for the target system. More peripherals such as UART, SPI, and GPIO can be added to make the system more complete. Verification will be expanded through dedicated self-checking testbenches and higher test coverage, while interrupt and exception handling can be introduced in later stages. Overall, the goal is to make the design more complete, scalable, reliable, and powerful as a RISC-V based SoC.

9. Conclusion

This project successfully implemented and functionally verified both Single-Cycle and Multi-Cycle 32-bit RISC-V (RV32I subset) processors. The Single-Cycle design provides a simple architecture in which each instruction is completed in one clock cycle, while the Multi-Cycle design shares hardware resources across multiple execution states to improve resource efficiency and support a shorter clock period. The complete system was extended with a bus interface, address decoder, and I2C Master peripheral, creating a modular structure in which the RISC-V processor can communicate with peripheral devices. Cycle-accurate ModelSim simulations confirmed successful execution of the supported instruction formats and verified the main system behavior. Overall, the project provides a solid foundation for developing a more complete and scalable RISC-V SoC.